Check for updates

Ranking Functions for Size-Change Termination

CHIN SOON LEE
Max-Planck-Institut für Informatik

This article explains how to construct a ranking function for any program that is proved terminating by *size-change analysis*.

The "principle of size-change termination" for a first-order functional language with well-ordered data is intuitive: A program terminates on all inputs, if every infinite call sequence (following program control flow) would imply an infinite descent in some data values. Size-change analysis is based on information associated with the subject program's call-sites. This information indicates, for each call-site, strict or weak data decreases observed as a computation traverses the call-site. The set *DESC* of call-site sequences for which the size-changes imply infinite descent is ω-regular, as is the set *FLOW* of infinite call-site sequences following the program flowchart. If $FLOW \subseteq DESC$ (a decidable problem), every infinite call sequence would imply infinite descent in a well-ordering—an impossibility—so the program must terminate.

This analysis accounts for termination arguments applicable to different call-site sequences, without indicating a ranking function for the program's termination. In this article, it is explained how one can be constructed whenever size-change analysis succeeds. The constructed function has an unexpectedly simple form; it is expressed using only min, max, and lexicographic tuples of parameters and constants. In principle, such functions can be tested to determine whether size-change analysis will be successful. As a corollary, if a program verified as terminating performs only multiply recursive operations, the function that it computes is multiply recursive.

The ranking function construction is connected with the determinization of the Büchi automaton for *DESC*. While the result is not practical, it is of value in addressing the scope of size-change reasoning. This reasoning has been applied broadly, in the analysis of functional and logic programs, as well as term rewrite systems.

Categories and Subject Descriptors: D.2.4 [**Software Engineering**]: Software/Program Verification; F.3.1 [**Logics and Meanings of Programs**]: Specifying and Verifying and Reasoning about Programs

General Terms: Theory

Additional Key Words and Phrases: ω-Automaton, determinization, multiple recursion, ranking function, size-change termination, termination analysis

Authors' address: Max-Planck-Institut für Informatik, Saarbrücken, Germany; email: cslee@asteria.com.sg.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.
© 2009 ACM 0164-0925/2009/04-ART10 $5.00
DOI 10.1145/1498926.1498928 http://doi.acm.org/10.1145/1498926.1498928

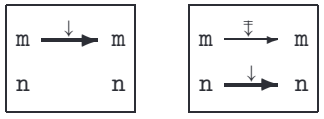
ACM Reference Format:
Lee, C. S. 2009. Ranking functions for size-change termination. ACM Trans. Program. Lang. Syst. 31, 3, Article 10 (April 2009), 42 pages. DOI = 10.1145/1498926.1498928
http://doi.acm.org/10.1145/1498926.1498928

1. INTRODUCTION

Size-change analysis was formulated as a termination analysis for first-order functional programs [Lee et al. 2001], but can be applied, with appropriate supporting techniques, to higher-order programs [Jones and Bohr 2004], logic programs [Codish et al. 2005], and term rewrite systems [Thiemann and Giesl 2003]. The logical reasoning behind the analysis is best illustrated with an example. Consider any call to the Ackermann function in Example 1.1 with a pair (m, n) of natural number values. Observe that the parameters `m` and `n` assume only natural number values during program execution. Thus, these values are well-ordered under the standard ordering of the natural numbers.

Example 1.1. (Ackermann)

```
a(m,n) = if m=0 then n+1 else
         if n=0 then 1a(m-1,1) else
         2a(m-1,3a(m,n-1))
```



$G_1,\ G_2$ $\qquad$ G_3

There are three call-sites in the program, labeled 1, 2, and 3. Each call-site c is associated with a *size-change graph* G_c, a digraph with distinguished source and target nodes that reflects data decreases observed as a computation traverses the call-site. Informally, each arc $x \xrightarrow{\gamma} y$ in G_c represents a relation between the value of parameter x before the call and the value passed for parameter y of the called function; $\gamma = \downarrow$ indicates a strict decrease, while $\gamma = \overset{=}{\downarrow}$ indicates that the prior value is not increased (i.e., it is *weakly* decreased). For example, G_3 asserts that in any call due to call-site 3, the value of `m` before the call is greater than or equal to the value of `m` after the call, while the value of `n` before the call is strictly greater than the value of `n` after the call.

Now consider a hypothetical infinite chain of (recursive) calls resulting from the initial call to function `a`. The corresponding sequence of call-sites $cs = c_1c_2\ldots$ (where each $c_i \in \{1, 2, 3\}$) induces a sequence of size-change graphs $G_{c_1}G_{c_2}\ldots$. If the call-site sequence ends in just 3's, then the graph sequence ends in just G_3's. The *concatenation* of these graphs, called the *multipath for* cs, is obtained by identifying the target nodes of each graph with the source nodes of the next one in the sequence, as depicted at the top of Figure 1. It exhibits infinite descent in a value of `n`, as reflected by a path from a node labeled `n` containing infinitely many $\downarrow$ arcs. If the call-site sequence does *not* end in just 3's, it has to contain infinitely many occurrences of 1 or 2; in this case, the

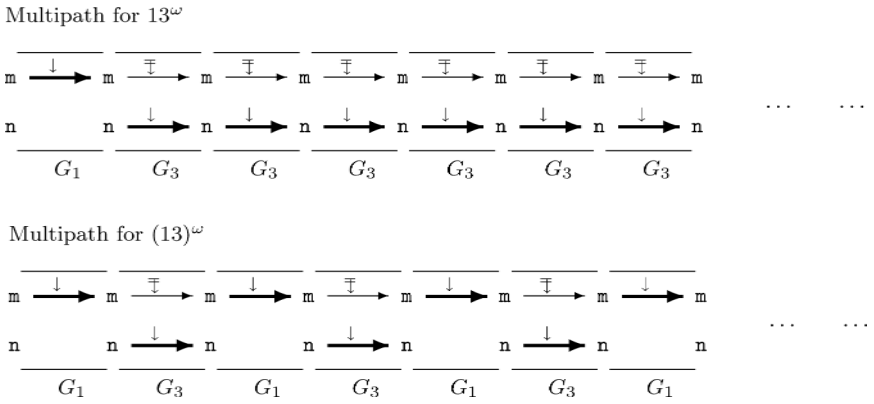


Fig. 1. Multipaths for Example 1.1 (Ackermann).

corresponding multipath exhibits infinite descent in the initial value of m (refer to the bottom of Figure 1).

The "principle of size-change termination" for a first-order functional language with well-ordered data can be expressed as follows.

> Infinite program execution is impossible, if every infinite call sequence would give rise to an infinitely decreasing value sequence, according to the size-change graphs.

It implies that the example above is terminating.

The noteworthy features of size-change analysis can be explained with reference to this example.

(1) *Automation*: It is well-suited for automation. Program analysis can be used to extract size-change graphs from the program, and it turns out to be decidable whether information in the graphs implies that every infinite call sequence would cause infinite descent in some data values [Lee et al. 2001].
(2) *Efficiency*: The size-change graphs seen above are *stratifiable*, and can be analyzed efficiently (in quadratic time) [Ben-Amram and Lee 2007]. If one is restricted to the information contained in stratifiable graphs, it simply does not make sense to search for lexicographic descent by brute force. In practice, size-change graphs tend to be simple (even if not necessarily stratifiable), which leads to an efficient analysis [Thiemann and Giesl 2003].
(3) *Generality*: Size-change analysis handles simple lexicographic descent (as in the above example) and simple multiset descent [Thiemann and Giesl 2003]. However, it is not immediately clear what the scope of the analysis is. That is the subject matter of this article.

One way to assess the scope of the analysis is to describe a *ranking function* for the subject program's termination whenever the analysis is successful. In this article, a ranking function ρ maps program states to elements of a well-ordering $(W, \leq_W)$ such that if the evaluation of a program function f with parameter values $\vec{v}$ leads to a call of function g with parameters $\vec{u}$,

then $\rho(g, \vec{u}) <_W \rho(f, \vec{v})$, where $<_W$ is the strict part of $\leq_W$. The well-ordering indicates an induction principle that can support the size-change reasoning. At the same time, a simple description of ρ is useful when comparing size-change analysis with other termination analyses. For the running example, take W to be pairs of natural numbers, ordered lexicographically, and let ρ map $(\mathtt{a}, (m, n))$ to the lexicographic pair $\langle m, n \rangle$.

Given a set of size-change graphs, size-change analysis accounts for any termination argument (based on the graphs) that is applicable to a call-site sequence. Consider the call-site sequence 13^ω (i.e., 1 followed by infinitely many 3's) for Example 1.1, which corresponds to the multipath at the top of Figure 1. The termination argument here is based on *some* value of n (but not the initial one). For the call-site sequence $(13)^\omega$ (refer to the bottom of Figure 1), the termination argument is based on the initial value of m. It is not immediately clear how this type of reasoning can be reconciled with ranking-function based reasoning. Indeed, there are examples of programs verified by size-change analysis, whose termination has so far *seemed* difficult to characterize with simple ranking functions, at least without recourse to some form of computation history [Ben-Amram 2002].

So how far does size-change termination (i.e., termination established by size-change analysis) extend beyond lexicographic descent? The answer is: Not very far! For any instance of size-change termination based on graphs with only strict decreases, a corresponding ranking function can be constructed using only min, max, and the function parameters. Generally, for *any* instance of size-change termination, a corresponding ranking function can be constructed using min, max, and lexicographic tuples of function parameters and constants. In principle, such functions can be tested to determine whether size-change analysis will succeed.

The ranking functions corresponding to size-change termination are of a particularly simple form; they are just not ones that are usually considered by other termination analyses.

Well-ordering of data values. The assumption of a well-ordering of the program's data values may seem artificial to the reader. For first-order functional programs that manipulate structured data, it is often some notion of *size* of a value, for example, the number of *cons* in a list, that is relevant to the termination of a recursion.

In practice, a *norm function* is used to assign each data value a size (some natural number). The arcs of a size-change graph are interpreted as relations between the sizes of parameter values. Thus, $x \stackrel{\downarrow}{\rightarrow} y$ signifies that the value in x has a greater size than the value passed for y, while $x \stackrel{\mp}{\rightarrow} y$ signifies that the value in x has a greater size or is the same size as the value passed for y. Size-change analysis remains sound under this interpretation. Of greater relevance to this article, the ranking function construction here remains valid by interpreting any reference to a function parameter as a reference to its size.

More generally, a node in a size-change graph can be used to denote some mapping of parameter values into a well-ordered set. Then any reference to the node in the ranking function construction should be interpreted as a reference

to the denoted value. The assumption of well-ordered data is made for the sake of simplicity.

1.1 Motivating Examples

This subsection contains examples of size-change termination. For each example, the size-change argument is juxtaposed with an argument based on ranking functions. While the presentation here is necessarily informal, hopefully it will be clear why a call to any function in each of the programs must terminate. The reader may like to review these examples after the formalization of size-change analysis.

Each example consists of a partially specified program. The ellipsis ... is used to indicate omitted code *not containing any function calls*. The omission is deliberate, to emphasize that the termination of the program follows already from the code that is shown. The "challenge" in each case is to provide a ranking function for the program's termination.

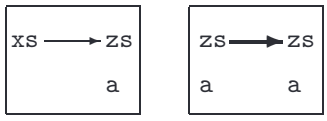
The semantics of a program (to be formalized) is based on the standard left-to-right call-by-value evaluation of a first-order functional language. The example programs operate on data values D built recursively from the natural numbers, the *empty list* "[ ]" and the *cons* operator ":" for lists. The data values are well-ordered by $\leq_D$ in a way that respects the standard ordering of the natural numbers and such that $v \leq_D (v : v')$ and $v' \leq_D (v : v')$. As usual, $<_D$ denotes the strict part of $\leq_D$. There are many possible definitions for $\leq_D$; its precise definition is not critical. The standard unary operators `hd`, `tl`, and $\bullet$ `- 1` are *destructive*, in the sense that applying any of them to a value v results in some $v' <_D v$. This implies that an unbounded number of applications of such operations to any value is *impossible*. Thus, attempting to take the head or tail of an empty list, or compute the predecessor of 0 should result in an error condition, preventing further program execution. Finally, program execution is assumed to be free from type errors.

Each call-site c in a program is associated with a size-change graph G_c. For a call-site c occurring in the definition of function f and representing a call to function g, write $G_c : f \to g$. Informally, G_c refers to a digraph with distinguished source and target nodes, labeled respectively by the parameters of f and g. An arc $x \stackrel{\gamma}{\to} y$ in the graph signifies that in any function call via call-site c, if v is the value in x before the call and u is the value passed for y, then $u \leq_D v$. Moreover, if $\gamma = \downarrow$, then $u \neq v$, thus $u <_D v$. In diagrams, $\downarrow$-labeled arcs are drawn with heavy lines, but the labels themselves are omitted to avoid clutter.

Example 1.2. (List reverse)

The only example of size-change termination seen so far (Example 1.1) is for a program with a single function. When multiple functions are involved, only call-site sequences respecting the program flowchart are considered in the size-change reasoning. The following program fragment (based on a standard algorithm for computing the reverse of a list), has two functions, `r1` and `r0`, and two call-sites, labeled 1 and 2. The size-change graphs corresponding to these call-sites, $G_1 : \texttt{r1} \to \texttt{r0}$ and $G_2 : \texttt{r0} \to \texttt{r0}$ are shown on the right.

```
r1(xs)   = 1r0(xs,[])
r0(zs,a) = if ... else
           2r0(tl zs,(hd zs):a)
```



$G_1 : \mathtt{r1} \to \mathtt{r0}$ $G_2 : \mathtt{r0} \to \mathtt{r0}$

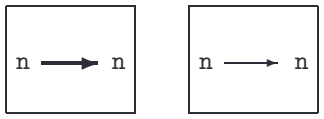
Clearly, the only infinite call-site sequence respecting program control flow is 12^ω. The arc $\mathtt{zs} \overset{\downarrow}{\to} \mathtt{zs}$ in G_2 shows that following this call-site sequence would cause infinite descent.

An "obvious" ranking function for this program's termination maps any program state of the form $(\mathtt{r1}, (xs))$ to $\langle 1, xs\rangle$ and any program state of the form $(\mathtt{r0}, (zs, a))$ to $\langle 0, zs\rangle$. The co-domain of this ranking function is D^2, ordered lexicographically. For convenience, such ranking functions will be defined function-wise. For the example, let $\rho_{\mathtt{r1}}(xs) = \langle 1, xs\rangle$ and $\rho_{\mathtt{r0}}(zs, a) = \langle 0, zs\rangle$.

Example 1.3. (Mutual recursion)

The example below shows that a descent is not required at every function call for size-change termination.

```
f0(n) = if ... else 1f1(n-1)
f1(n) = if ... else 2f0(n)
```



$G_1 : \mathtt{f0} \to \mathtt{f1}$ $G_2 : \mathtt{f1} \to \mathtt{f0}$

Any infinite call-site sequence respecting the program flowchart has to end with $(12)^\omega$. The graphs G_1 and G_2 show that following the call-site sequence would cause infinite descent in n.

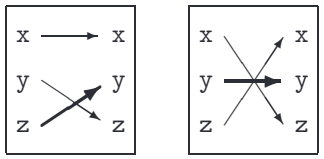
For a ranking function to show the program's termination, let $\rho_{\mathtt{f0}}(n) = \langle n, 0\rangle$ and $\rho_{\mathtt{f1}}(n) = \langle n, 1\rangle$. The ranking function has codomain D^2.

The program flowchart turns out not to be a real hurdle for constructing ranking functions corresponding to size-change termination, so the remaining examples focus on different forms of parameter-descent behavior over a single function.

Example 1.4. (Permuted arguments)

The parameters of program function p in the example below are "moved around" on each call.

```
p(x,y,z) = if ...
           1p(x,z-1,y) else
           2p(z,y-1,x)
```



G_1 G_2

Consider any infinite call-site sequence $cs = c_1c_2\ldots$, where each $c_i \in \{1, 2\}$. The claim is that the corresponding multipath (the concatenated sequence of size-change graphs $G_{c_1}G_{c_2}\ldots$) exhibits infinite descent. Figure 2 shows an initial segment of such a multipath. Observe that it contains exactly three maximal paths, representing three distinct dataflows. Since each G_{c_i} contributes

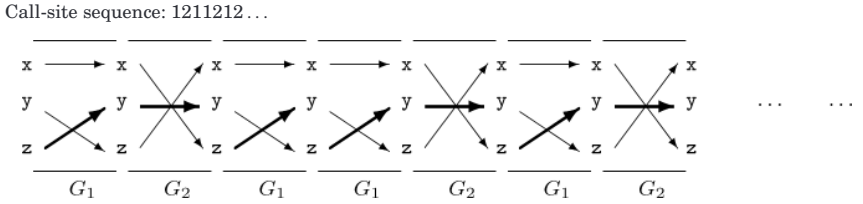


Fig. 2. A multipath for Example 1.4 (permuted arguments).

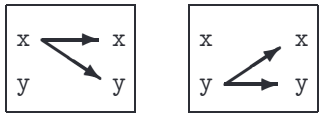
a ↓-labeled arc to some path, one of the paths has infinitely many ↓-labeled arcs.

A simple ranking function for this program's termination is $\rho_{\mathrm{p}}(x, y, z) = x + y + z$, with co-domain $\mathbb{N}$. However, allowing the use of sums in the definition of a ranking function is not helpful with the *general* problem of corresponding ranking functions to size-change termination.

On deeper reflection, the size-change argument is related to lexicographic descent. Let M and M' be the multisets of parameter values before and after a call respectively. Then both graphs imply that M' is obtained from M by replacing an element of M with a smaller one. Thus, $M' < M$ under the standard multiset ordering [Dershowitz and Manna 1979]. For well-ordered data, multiset descent is equivalent to sorting M and M' in descending order and comparing the resulting tuples lexicographically. A possibility for ρ_{p} is therefore given by $\rho_{\mathrm{p}}(x, y, z) = \mathit{sortdown}\langle x, y, z\rangle$, where *sortdown* rearranges the components of $\langle x, y, z\rangle$ in descending order. The co-domain of this ranking function is D^3.

Example 1.5. (Maximum descending)

```
mx(x,y) = if ...
          1mx(hd(tl x),tl(tl x)) else
          2mx(hd(tl y),tl(tl y))
```



G_1 G_2

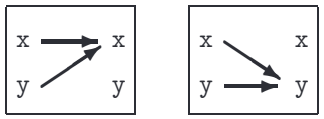
Consider any infinite call-site sequence for this example. If it ends in just 1's (resp. 2's), then G_1 (resp. G_2) shows that following this call-site sequence would cause infinite descent in x (resp. y). Otherwise, the call-site sequence has infinitely many 1's and 2's. Drop any initial occurrences of 2 and divide the remaining sequence into segments of the form $(1^*1)(2^*2)$. Consider the multipath for any such segment. A sequence of arcs connecting the first and final x in the multipath can be obtained this way: Follow any arcs from x to x across 1^*, then take the arc from x to y. Next, follow any arcs from y to y across 2^*, then take the arc from y to x. Following the entire call-site sequence would cause infinite descent in x.

A simple ranking function for the program's termination is $\rho_{\mathtt{mx}}(x, y) = \max\{x, y\}$, with co-domain D.

Example 1.6. (Minimum descending)

The size-change graphs seen so far are *fan-in free*; there is at most one arc ending at each target node. The following example shows graphs with fan-in.

```
mn(x,y) = if ...
          if x<y then 1mn(x-1,...)
          else 2mn(...,y-1)
```



G_1 G_2

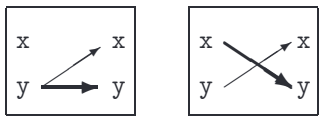
Observe that any infinite multipath contains an infinite path, since both source nodes in each size-change graph are connected to some target node. As every arc is labeled with $\downarrow$, this path has infinitely many $\downarrow$-labeled arcs.

A simple ranking function for the program's termination is $\rho_{\mathtt{mn}}(x, y) = \min\{x, y\}$, with co-domain D.

Example 1.7. (Mystery descent 2001)

This example was presented in Lee et al. [2001] and Thiemann and Giesl [2003] as "interesting" (perhaps because its ranking function confounded the authors).

```
m(x,y) = if ...
         1m(y,y-1) else
         2m(y,x-1)
```



G_1 G_2

Consider any infinite call-site sequence. If it ends in just 2's, then G_2 shows that following this call-site sequence would cause infinite descent in x or y (this is the same descent as in Example 1.4, seen earlier). Otherwise, the call-site sequence contains infinitely many 1's. Drop any initial occurrences of 2 and divide the remaining sequence into segments of the form $1(22)^*$ or $12(22)^*$. The size-change graphs show that the value of y is strictly decreased on following 1 or 12 and weakly decreased on following $(22)^*$.

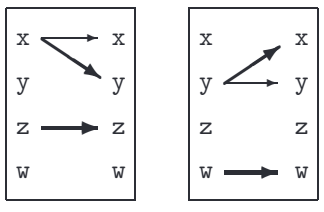
A possible (but awkward) ranking function for this program's termination is $\rho_{\mathtt{m}}(x, y) = \max\{2x - 1, 2y\}$ with co-domain $\mathbb{N}$. The reader may like to verify that for any $x, y, x', y' \geq 0$, $(x' \leq y$ and $y' < y)$ or $(x' \leq y$ and $y' < x)$ implies that $\max\{2x' - 1, 2y'\} < \max\{2x - 1, 2y\}$.

The ranking function does not directly reflect the size-change argument, as it is predicated on x and y holding natural number values.

Example 1.8. (Mystery descent 2003)

The final example demonstrates how size-change analysis applies different termination arguments across different subsets of call-site sequences.

```
m(x,y,z,w) = if ...
             1m(x,x-1,z-1,...) else
             2m(y-1,y,...,w-1)
```



G_1 G_2

Consider any infinite call-site sequence. If it ends in just 1's (resp. 2's), the size-change graph G_1 (resp. G_2) shows that following this call-site sequence would cause infinite descent in z (resp. w). Otherwise, the call-site sequence contains infinitely many 1's and 2's. Drop any initial occurrences of 2, and divide the remaining sequence into segments of the form $(1^*1)(2^*2)$. Argue as in Example 1.5 that following any such segment would cause a descent in x.

What is a ranking function for this program's termination?

It has been pointed out to the author that for Example 1.8 and similar programs, any tuple of linear functions over the parameters will not be suitable; even a tuple of functions constructed from linear operations, minimums, and maximums will not do the job. If forming lexicographic tuples of minimums and maximums does not produce suitable ranking functions, how about taking minimum and maximum *over* lexicographic tuples?

Claim: $\rho_{\mathtt{m}}(x, y, z, w) = \max\{\langle x, z\rangle, \langle y, w\rangle\}$, with codomain D^2, is a ranking function that can be used to show the termination of Example 1.8, where the maximum is based on lexicographic comparison.

PROOF. In a call from program state $(\mathtt{m}, (x, y, z, w))$ to $(\mathtt{m}, (x', y', z', w'))$, either:

(1) $x' \leq x$ and $y' < x$ and $z' < z$ (if the call is due to call-site 1), or
(2) $x' < y$ and $y' \leq y$ and $w' < w$ (if the call is due to call-site 2).

In the first case, $\langle x', z'\rangle < \langle x, z\rangle$ since $x' \leq x$ and $z' < z$, and $\langle y', w'\rangle < \langle x, z\rangle$ since $y' < x$. It follows that $\rho_{\mathtt{m}}(x', y', z', w') = \max\{\langle x', z'\rangle, \langle y', w'\rangle\} < \langle x, z\rangle$. But $\langle x, z\rangle \leq \max\{\langle x, z\rangle, \langle y, w\rangle\} = \rho_{\mathtt{m}}(x, y, z, w)$. Therefore, the value of $\rho_{\mathtt{m}}$ after the call is strictly smaller than its value before the call. The same claim holds for the second case by symmetry. □

This article will prove, by construction, that *any* instance of size-change termination corresponds to a ranking function of a similar form. In general, each ρ_f will be a (single) minimum over maximums of lexicographic tuples containing parameters and constants.

This means that Example 1.4 (permuted arguments) and 1.7 (mystery descent 2001) can be shown to terminate by such ranking functions. In fact, for these examples, the ranking functions can be of the same form as Example 1.8, comprising just a maximum over lexicographic tuples. The reader is invited to work them out at this point. Solutions will be given in Section 5.

Implications of the result. The ranking function corresponding to an instance of size-change termination has codomain D^k, ordered lexicographically. The value of k determines the induction principle able to support the size-change reasoning. An easy corollary attesting to the limits of size-change analysis is: If a program is verified as terminating by size-change analysis and performs only multiply recursive operations, then the function computed is multiply recursive [Ben-Amram 2002].

A space of possible ranking functions for a size-change termination problem is easily described. The only variables are the length of the tuples to use and the

number of constants the tuples may contain. Suitable values for these variables can be worked out from the number of size-change graphs and the maximum number of nodes in a graph. In principle, this space of functions can be tested to determine whether size-change analysis will succeed, although doing so is not practical.

1.2 Structure of the Article

The next section formalizes size-change termination analysis for a prototypical functional subject language. Sections 3 and 4 work towards a procedure that constructs, for a given instance of size-change termination, a corresponding ranking function. Section 3 considers the problem restricted to subject programs with a single function. It contains the main technical developments. The following section then accounts for the flowchart. Section 5 contains some concluding remarks.

2. SIZE-CHANGE TERMINATION ANALYSIS

This section introduces the subject language, explains the role of *ranking functions* in proving program termination, and formalizes *size-change termination analysis*.

2.1 The Subject Language

The subject language is a standard minimal first-order functional language. Programs in this language are generated by the following grammar.

$$
\begin{array}{llll}
\mathit{Prg} & \ni & p & ::= d_1 \ \ldots \ d_N \\
\mathit{Dfn} & \ni & d & ::= f(x_1, \ldots, x_K) = e_f \\
\mathit{Exp} & \ni & e & ::= x \mid \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \mid op(e_1, \ldots, e_K) \mid {}^{c} f(e_1, \ldots, e_K) \\
\mathit{Par} & \ni & x & ::= \text{parameter identifier} \\
\mathit{Fun} & \ni & f & ::= \text{function identifier} \\
\mathit{Op} & \ni & op & ::= \text{primitive operator}
\end{array}
$$

A subject program p consists of a nonempty set of *function definitions*. A definition has the form $f(x_1, \ldots, x_K) = e_f$, where e_f is called the *body* of f. The number $ar(f)$, called the *arity* of f, is K, which is assumed to be positive. Let $Param(f)$ denote the set $\{x_1, \ldots, x_K\}$, the formal parameters of f. Operators that take no arguments are called *constants*. It is assumed that operators, parameter identifiers, and function identifiers are pair-wise disjoint. Refer to the superscript c labeling the expression ${}^{c} f(e_1, \ldots, e_K)$ as a *call-site*, and write $c : g \rightarrow f$ if c occurs in the body of g. For any subject program, the finite set of call-sites is denoted *Sites*. Let *Par*, *Fun*, and *Op* be respectively the parameters, functions, and operators occurring in the program.

It will be assumed that the following hold. Each member of *Fun* is defined exactly once. Each member of *Par* is introduced exactly once (on the left-hand side of a definition). Any use of a variable is in scope; that is, any variable occurring on the right-hand side of a definition also occurs on the left. Finally, there are no arity problems, that is, the number of formal and actual arguments for a function f is consistent throughout the program.

Program semantics. A semantics for the subject program provides a means to evaluate any program function f given an assignment of values to its parameters. This is taken to mean evaluating the *body* of f.

Expression evaluation is based on a standard left-to-right call-by-value strategy, with explicit error handling. Formally, let D be the set of data values, and let $D^\sharp = D \cup \{\bot, \mathit{Err}\}$. Use the flat lattice $(D^\sharp, \sqsubseteq)$ as the semantic domain.[1] Note that the semantic ordering $\sqsubseteq$ is distinct from the ordering $\leq_D$ on D discussed previously; the two orderings should not be confused. The precise definition of D is not important for the result in this article, but it is supposed that D is infinite and forms a well-ordering with $\leq_D$. The strict part of $\leq_D$ is denoted $<_D$. For $n \in \mathbb{N}$, write $(n)_D$ for the n-th smallest value of D under $\leq_D$.

The evaluation of a subexpression e in the body of f with respect to value assignment $\vec{v} \in D^{ar(f)}$ is obtained by applying the least fixed point of a functional $\mathcal{F}$, defined shortly, to e and $\vec{v}$. A few auxiliary functions are needed to define $\mathcal{F}$. The injection $\mathit{lift} : D \to D^\sharp$ embeds D into $D^\sharp$. Function $\mathcal{O} : Op \to D^* \to D^\sharp$, used for interpreting operations on D values, is not defined, but it is assumed that $\mathcal{O}[\![op]\!]\vec{v} \neq \bot$, i.e., any primitive operation must terminate. The subject language is untyped, so 0 and 1 are conveniently used to represent the boolean values of *false* and *true* respectively. With such an encoding in mind, the semantic function *cnd*, used for interpreting `if`-expressions, is defined as follows.

$$\mathit{cnd}\ w_1\ w_2\ w_3 = \begin{cases} w_1, & \text{if } w_1 = \bot, \\ w_2, & \text{if } w_1 = \mathit{lift}\ 1 \text{ (signifying } \mathit{true}), \\ w_3, & \text{if } w_1 = \mathit{lift}\ 0 \text{ (signifying } \mathit{false}), \\ \mathit{Err}, & \text{otherwise.} \end{cases}$$

The function $\mathit{strictapp} : (D^* \to D^\sharp) \to (D^{\sharp *} \to D^\sharp)$ below is for handling nonterminating or erroneous subcomputations in operations and function calls. For any tuple $\vec{v} = (v_1, \ldots, v_K)$, write $(\vec{v})_i$ for v_i, the i-th element of $\vec{v}$.

$$\mathit{strictapp}\ \psi\ \vec{w} = \begin{cases} \psi\ \vec{v}, & \text{if } (\vec{w})_i = \mathit{lift}\ (\vec{v})_i \text{ for each } i, \text{ else} \\ (\vec{w})_i, & \text{where } i \text{ is the least index with } (\vec{w})_i \in \{\bot, \mathit{Err}\}. \end{cases}$$

Finally, the functional $\mathcal{F}$ is defined. Its least fixed point $\mathcal{E} : \mathit{Exp} \to D^* \to D^\sharp$ exists since $\mathcal{F}$ is monotonic in the extension of $\sqsubseteq$ to the domain of $\mathcal{E}$.

$$\begin{array}{ll} \lambda\, \mathcal{E}\ e\ \vec{u}\ \cdot \text{ case } e \text{ of} & \\ \quad x \in \mathit{Param}(f) & : \mathit{lift}\ (\vec{u})_i, \text{ where } x \text{ is the } i\text{-th parameter of } f \\ \quad \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 & : \mathit{cnd}\ (\mathcal{E}[\![e_1]\!]\vec{u})\ (\mathcal{E}[\![e_2]\!]\vec{u})\ (\mathcal{E}[\![e_3]\!]\vec{u}) \\ \quad op(e_1, \ldots, e_K) & : \mathit{strictapp}\ \ (\mathcal{O}[\![op]\!])\ \ (\mathcal{E}[\![e_1]\!]\vec{u}, \ldots, \mathcal{E}[\![e_K]\!]\vec{u}) \\ \quad {}^c f(e_1, \ldots, e_K) & : \mathit{strictapp}\ \ (\mathcal{E}[\![e_f]\!])\ \ (\mathcal{E}[\![e_1]\!]\vec{u}, \ldots, \mathcal{E}[\![e_K]\!]\vec{u}) \end{array}$$

The evaluation of e from the body of f with respect to $\vec{v} \in D^{ar(f)}$ is $\mathcal{E}[\![e]\!]\vec{v}$.

Definition 2.1. (Termination) Let $f \in \mathit{Fun}$ and $\vec{v} \in D^{ar(f)}$. Refer to $(f, \vec{v})$ as a *program state of* p. It is *divergent* if $\mathcal{E}[\![e_f]\!]\vec{v} = \bot$.

Let St denote the set of all program states of p. Program p is *terminating* if St contains no divergent states.

[1]This means that for all $w \in D^\sharp$, $w' \sqsubseteq w$ just when $w' = w$ or $w' = \bot$.

State transitions. The termination arguments considered in this article are based on *function calls*, so it makes sense to formalize the possible function calls of the subject program as a transition system on the program states St and express the termination problem in terms of the transition system.

More terminology: A program state $s \in St$ has the form $(f, \vec{v})$. If $x \in Param(f)$ is the i-th parameter of f, then the *value of x in s* is $(\vec{v})_i$.

Definition 2.2. Let $Calls = St \times Sites \times St$.

(1) A *transition relation* $\rightarrow$ is any subset of $Calls$. Refer to an element of $\rightarrow$ as a *state transition*, and write $s \xrightarrow{c} s'$ for $(s, c, s') \in \rightarrow$.
(2) An *infinite $\rightarrow$-chain* is an infinite sequence of the form $s_0 c_1 s_1 c_2 s_2 \ldots$, also written as $s_0 \xrightarrow{c_1} s_1 \xrightarrow{c_2} s_2 \ldots$, such that $s_{t-1} \xrightarrow{c_t} s_t$ for $t > 0$.
(3) The relation $\rightarrow$ is *well-founded* if there does not exist any infinite $\rightarrow$-chain.

The operator $\mathcal{T}$ below is used to define the transition relation of the subject program. It maps a program expression e and a tuple of values $\vec{u}$ to a subset of $Calls$. This subset captures what are intuitively the local function calls observed during evaluation of e with respect to $\vec{u}$. In the following auxiliary functions, $w, (\vec{w})_i \in D^\sharp$ and $C_i, (\vec{C})_i \subseteq Calls$.

$$select\ w\ (C_1, C_2, C_3) = \begin{cases} C_1 \cup C_2, & \text{if } w = lift\ 1 \text{ (signifying } true\text{)}, \\ C_1 \cup C_3, & \text{if } w = lift\ 0 \text{ (signifying } false\text{)}, \\ C_1, & \text{otherwise.} \end{cases}$$

$$strictarg\ \vec{w}\ \vec{C} = \bigcup\{(\vec{C})_i \mid \forall j < i \cdot (\vec{w})_j = lift\ v \text{ for some } v\}$$

For call-site $c : g \rightarrow f$ and $\vec{u}, \vec{v} \in D^*$, let $call(\vec{u}, c, \vec{v}) = ((g, \vec{u}), c, (f, \vec{v}))$. The value of $\mathcal{T}[\![e]\!]\vec{u}$ is defined recursively. First, given $e_1, e_2, \ldots$, let $\vec{w}$ and $\vec{C}$ be such that $(\vec{w})_i = \mathcal{E}[\![e_i]\!]\vec{u}$ and $(\vec{C})_i = \mathcal{T}[\![e_i]\!]\vec{u}$. Perform case analysis on the form of e:

—If $e = x$, then $\mathcal{T}[\![e]\!]\vec{u} = \{\}$.
—If $e =$ `if` e_1 `then` e_2 `else` e_3, $\mathcal{T}[\![e]\!]\vec{u} = select\ (\vec{w})_1\ \vec{C}$.
—If $e = op(e_1, \ldots, e_K)$, $\mathcal{T}[\![e]\!]\vec{u} = strictarg\ \vec{w}\ \vec{C}$.
—If $e = {}^c f(e_1, \ldots, e_K)$, $\mathcal{T}[\![e]\!]\vec{u} = strictarg\ \vec{w}\ \vec{C} \cup \{call(\vec{u}, c, \vec{v}) \mid \forall i \cdot lift\ (\vec{v})_i = (\vec{w})_i\}$.

Definition 2.3. For the subject program p, let $\rightarrow_p\ = \bigcup\{\mathcal{T}[\![e_f]\!]\vec{v} \mid (f, \vec{v}) \in St\}$. This relation is called the *transition relation of p*. Refer to any (s, c, s') in $\rightarrow_p$ as a *state transition of p*, and write $s \xrightarrow{c}_p s'$.

The following claims formalize the intuition that the subject program's execution is finite just when no infinite chain of function calls is possible. Their proofs are straightforward, but are included in the appendix for completeness.

LEMMA 2.4. *Let $s = (f, \vec{v}) \in St$ and e be a subexpression in the body of f. Evaluation $\mathcal{E}[\![e]\!]\vec{v} = \perp$ just when $\mathcal{T}[\![e]\!]\vec{v}$ contains some (s, c, s') with s' divergent.*

Intuitively, the evaluation of an expression diverges just when it makes a call that diverges. The lemma supports the following.

LEMMA 2.5. *If a program state is divergent, there exists an infinite* $\rightarrow_p$*-chain that starts with it.*

COROLLARY 2.6. *If* $\rightarrow_p$ *is well-founded, then* p *is terminating.*

In fact, the converse of Lemma 2.5 and Corollary 2.6 also hold, as explained in the appendix. Thus, the termination of subject program p is formally reducible to the well-foundedness of its transition relation. It is this well-foundedness problem that is addressed by size-change analysis.

Definition 2.7. Let $\rightarrow$ be *any* transition relation and $(W, \leq_W)$ a well-ordering. A $\rightarrow$*-ranking function into* $(W, \leq_W)$ is a mapping $\rho : St \rightarrow W$ such that $\rho(s') <_W \rho(s)$ whenever $s \overset{c}{\rightarrow} s'$, where $<_W$ is the strict part of $\leq_W$.

Recall that $St = Fun \times D^*$. In the remainder of the article, the notation $\rho_f(s)$ will be used to indicate that ρ is being applied to a state of the form $(f, \vec{v})$, although in examples, it is customary to apply ρ_f directly to the argument tuple $\vec{v}$.

Clearly, the existence of a $\rightarrow_p$-ranking function implies the well-foundedness of $\rightarrow_p$ and consequently the termination of program p. A straightforward approach to termination analysis is to derive somehow such a ranking function.

2.2 Size-Change Graphs and Their Semantics

In this subsection, the *size-change graph* is formally defined, and it is explained how a set of graphs is seen as specifying an over-approximation of $\rightarrow_p$.

Definition 2.8. A *size-change graph* for call-site $c : g \rightarrow f$ is formally a triple consisting of *source function* g, *target function* f, and a digraph G_c, described below. The triple is written as $G_c : g \rightarrow f$ or $g \overset{G_c}{\rightarrow} f$. When g and f are unimportant or clear from context, they may be omitted.

The nodes of G_c consists of two partitions: a set of *source nodes* labeled by $Param(g)$ and a set of *target nodes* labeled by $Param(f)$. The arcs of G_c, called *size-change arcs*, are directed from the source nodes to the target nodes. Each arc is labeled uniquely with either $\downarrow$ or $\ddagger$. An arc is therefore specified by a triple in $Param(g) \times \{\downarrow, \ddagger\} \times Param(f)$. Write $y \overset{\gamma}{\rightarrow} x \in G_c$ if G_c has an arc from source y to target x labeled with size-change γ. The arc is *strict* if $\gamma = \downarrow$, else *nonstrict*.

Semantically, a size-change arc signifies a relationship between a pair of data values. Recall that the data domain D forms a well-ordering with the relation $\leq_D$, whose strict part is $<_D$. A size-change arc from y to x in G_c asserts that for $s \overset{c}{\rightarrow}_p s'$, the value u of y in s is related to the value v of x in s'. If the arc is strict, $v <_D u$, else $v \leq_D u$. This interpretation leads to a natural notion of what it means for a set $\mathcal{G}$ of size-change graphs to be *safe* for transition relation $\rightarrow$.

Definition 2.9. (Safety of $\mathcal{G}$) Let $\rightarrow$ be any transition relation, and let $\mathcal{G}$ contain a size-change graph G_c for each call-site c.

(1) Let $c : g \rightarrow f$ and suppose that $s \overset{c}{\rightarrow} s'$ for $s = (g, \vec{u})$ and $s' = (f, \vec{v})$. Now consider $G_c : g \rightarrow f$, the size-change graph in $\mathcal{G}$ corresponding to call-site

c. Let x be the i-th parameter of f and y the j-th parameter of g. A size-change arc $y \stackrel{\downarrow}{\rightarrow} x \in G_c$ is *safe for* (s, c, s') *of* $\rightarrow$ if $(\vec{v})_i <_D (\vec{u})_j$. A size-change arc $y \stackrel{\mp}{\rightarrow} x \in G_c$ is *safe for* (s, c, s') *of* $\rightarrow$ if $(\vec{v})_i \leq_D (\vec{u})_j$.

(2) A size-change arc of G_c is safe for $\rightarrow$ if it is safe for every (s, c, s') of $\rightarrow$.

(3) The graph $G_c : g \rightarrow f$ is safe for $\rightarrow$ if every arc of G_c is safe for $\rightarrow$.

(4) The set $\mathcal{G}$ is safe for $\rightarrow$ if each graph of $\mathcal{G}$ is safe for $\rightarrow$.

Call a unary operator *destructive* if every value that it returns is strictly smaller than its input. More formally, *op* is destructive if $\mathcal{O}[\![op]\!]u = lift\ v$ implies that $v <_D u$. For example, `tl` and `• - 1` are destructive.

The following is a simple and formally justifiable way to construct a $\mathcal{G}$ that is safe for $\rightarrow_p$. Let $f(e_1, \ldots, e_K)$ be the expression labeled by call-site c. Consider each argument expression e_i in turn. Let x be the i-th parameter of f. If e_i is the application of one or more destructive operators to a parameter y, for example, y `- 1` or `tl (tl y)`, include the arc $y \stackrel{\downarrow}{\rightarrow} x$ in G_c, and if e_i is just the parameter y, include the arc $y \stackrel{\mp}{\rightarrow} x$ in G_c.

A syntactic analysis like this is good enough to produce all the size-change graphs seen in Section 1 *except* the ones for Example 1.6. To obtain those graphs, it is necessary to analyze the conditions of `if`-expressions. In general, global program analysis may be required to deduce a relation that holds between an argument expression and an input parameter [Manolios and Vroon 2006].

A safe set of size-change graphs $\mathcal{G}$ is an over-approximation of $\rightarrow_p$ in the following sense. Let $\rightarrow_{\mathcal{G}}$ denote the maximal subset of *Calls* for which $\mathcal{G}$ is safe. Then $\mathcal{G}$ can be identified with the transition relation $\rightarrow_{\mathcal{G}}$. Clearly, $\mathcal{G}$ is safe for $\rightarrow_p$ just when $\rightarrow_p$ is a subset of $\rightarrow_{\mathcal{G}}$.

Refer to any (s, c, s') in $\rightarrow_{\mathcal{G}}$ as a $\mathcal{G}$-transition, and write $s \stackrel{c}{\rightarrow}_{\mathcal{G}} s'$. Note that $s \stackrel{c}{\rightarrow}_{\mathcal{G}} s'$ if and only if G_c is safe for (s, c, s').

The SCT condition. Size-change termination analysis is appealing because it is often easy to derive a safe $\mathcal{G}$ that contains sufficient information to prove the termination of the subject program p *and* because the problem of whether $\rightarrow_{\mathcal{G}}$ is well-founded is decidable. Clearly, the well-foundedness of $\rightarrow_{\mathcal{G}}$ implies the well-foundedness of $\rightarrow_p$, which implies the termination of p by Corollary 2.6.

One way to establish the well-foundedness of $\rightarrow_{\mathcal{G}}$ is to look for a $\rightarrow_{\mathcal{G}}$-ranking function, a mapping ρ from the program states *St* to elements of a well-ordering $(W, \leq_W)$ such that if $s \stackrel{c}{\rightarrow}_{\mathcal{G}} s'$, then $\rho(s') <_W \rho(s)$. However, the problem can be tackled without any reference to ranking functions.

Definition 2.10. (Multipaths and threads) Let $\mathcal{G}$ contain a size-change graph G_c for each call-site c.

(1) Call a (finite or infinite) call-site sequence $cs = c_1 c_2 \ldots$ *control-flow legal* if for some sequence $f_0, f_1, f_2, \ldots$ of elements of *Fun*, each $c_t : f_{t-1} \rightarrow f_t$.

(2) A $\mathcal{G}$-*multipath* of a control-flow legal call-site sequence $cs = c_1 c_2 \ldots c_t \ldots$ is a sequence of the form:

$$\mathcal{M}_{cs} = f_0 \overset{G_{c_1}}{\rightarrow} f_1 \overset{G_{c_2}}{\rightarrow} f_2 \ldots \overset{G_{c_t}}{\rightarrow} f_t \ldots$$

where $f_{t-1} \overset{G_{c_t}}{\rightarrow} f_t$ is in $\mathcal{G}$, for each $t > 0$.
The multipath is usually regarded a digraph. The node set of this digraph consists of partitions, one for each $t \geq 0$; the nodes within partition t are labeled by $Param(f_t)$. There is a γ-labeled arc in the digraph from node y in partition $(t-1)$ to node x in partition t just when $y \overset{\gamma}{\rightarrow} x \in G_{c_t}$. For a pictorial representation, see Figure 1 or Figure 2.

(3) A path in $\mathcal{M}_{cs}$ (regarded as a digraph) is called a *thread*. A thread ϑ does not have to start at the beginning of the multipath, i.e., from a node in partition 0. Suppose that it starts from a node in partition d. Then it has the form:

$$\vartheta = x_0 \overset{\gamma_1}{\rightarrow} x_1 \overset{\gamma_2}{\rightarrow} x_2 \ldots \overset{\gamma_t}{\rightarrow} x_t \ldots$$

where $x_{t-1} \overset{\gamma_t}{\rightarrow} x_t \in G_{c_{d+t}}$, for each $t > 0$.
The thread is *maximal* if it is a maximal path in $\mathcal{M}_{cs}$ and *complete* if it starts at the beginning of the multipath and is as long as the multipath. It is *descending* if it contains a strict arc and *has infinite descent* if it contains infinitely many strict arcs.

(4) Multipath $\mathcal{M}_{cs}$ *has infinite descent* if it has a thread with infinite descent.

Since the graph in $\mathcal{G}$ for call-site c represents any $s \overset{c}{\rightarrow}_{\mathcal{G}} s'$, a $\mathcal{G}$-multipath can be seen as representing a set of $\rightarrow_{\mathcal{G}}$-chains. The notion of *unrealizable* $\rightarrow_{\mathcal{G}}$-chains motivates the following.

Definition 2.11. Let $\mathcal{G}$ contain a size-change graph G_c for each call-site c. Then $\mathcal{G}$ satisfies the SCT property (or simply, *is* SCT) if every infinite $\mathcal{G}$-multipath has infinite descent.

In the remainder of this section, it will be shown that the SCT property is decidable and that $\mathcal{G}$ is SCT *just when* $\rightarrow_{\mathcal{G}}$ is well-founded. Consequently, it can be decided whether $\rightarrow_{\mathcal{G}}$ is well-founded.

If $\mathcal{G}$ is safe for $\rightarrow_p$, the transition relation of the subject program p, then $\rightarrow_{\mathcal{G}}$ is an over-approximation of $\rightarrow_p$. A successful verification of the well-foundedness of $\rightarrow_{\mathcal{G}}$ implies the well-foundedness of $\rightarrow_p$, hence the termination of p.

2.3 Deciding SCT via ω-Automata

One way to decide whether $\mathcal{G}$ satisfies the SCT property involves manipulating ω-automata. These are like finite state automata, but accept infinite words instead of finite words. Below, some basic theory on two types of ω-automata, the Büchi automaton and the Rabin automaton, is reviewed.

Definition 2.12. (Büchi automata)

(1) A *Büchi automaton* (BA) is a tuple $\mathcal{B} = (Q, \Sigma, q_0, \Delta_Q, A_Q)$, where Q is a finite set called *states*, Σ is a finite set called the *alphabet*, $q_0 \in Q$ is the

initial state, and $A_Q \subseteq S$ is the set of *accepting states*. The *transition relation* is a set $\Delta_Q \subseteq Q \times \Sigma \times Q$. The elements of this set are called *transitions*.

(2) The transition relation Δ_Q is deterministic if for every $r \in Q$ and $a \in \Sigma$, there is at most one $r' \in Q$ such that $(r, a, r') \in \Delta_Q$. An automaton is deterministic if its transition relation is deterministic.

(3) A *run of* $\mathcal{B}$ on a (finite or infinite, but nonempty) word $w = a_1 a_2 \ldots$ is a sequence of states $\varrho = r_0 r_1 r_2 \ldots$ such that $(r_{t-1}, a_t, r_t) \in \Delta_Q$ for each $t > 0$. The word w is said to *label* ϱ. It is also called a *labeling of* ϱ. The run ϱ *enters an accepting state* if $r_t \in A_Q$ for some $t > 0$.
By convention, a run has at least two states. For $t_0 < t_1$, call $r_{t_0} \ldots r_{t_1}$ a *subrun* of ϱ. For $t_0 < t_1 < t_2$, the subruns $r_{t_0} \ldots r_{t_1}$ and $r_{t_1} \ldots r_{t_2}$ are *consecutive*.

(4) Denote by $inf(\varrho)$ the set of states occurring infinitely in ϱ.

(5) The run ϱ is *tail-accepting* if $inf(\varrho) \cap A_Q \neq \{\}$; that is, some accepting state occurs infinitely in ϱ. It is *accepting* if it is tail-accepting and starts with q_0.

(6) The set of words $L_\omega(\mathcal{B}) = \{w \mid$ some run on w is accepting$\}$ is called the *language accepted by* $\mathcal{B}$.
The set of all infinite words is denoted Σ^ω. A subset L of Σ^ω is called *ω-regular* if $L = L_\omega(\mathcal{B})$ for some (not necessarily deterministic) BA $\mathcal{B}$.

There exists a nondeterministic BA whose language is not accepted by *any* deterministic BA, which explains why, in the definition of ω-regularity, it is emphasized that the BA is not necessarily deterministic. However, any nondeterministic BA can be determinized as a *Rabin automaton* (RA). In other words, given any BA, it is possible to build an RA with a deterministic transition relation that accepts the same language [Safra 1988; Muller and Schupp 1995; Althoff et al. 2005].

Definition 2.13. (Rabin automata)

(1) A *Rabin automaton* (RA) is a tuple $\mathcal{R} = (R, \Sigma, q_0, \Delta_R, \Omega_R)$ where the first four components are the same as for a BA.
The acceptance condition is specified by Ω_R, which is a finite set of *acceptance pairs*. An acceptance pair has the form (E, F), where E, called a *miss set*, and F, called a *hit set*, are subsets of R.
The automaton is deterministic if its transition relation is deterministic.

(2) A *run of* $\mathcal{R}$ on an infinite word $w = a_1 a_2 \ldots$ is defined as for a BA.
A run ς is *tail-accepting* if for some acceptance pair (E, F), $inf(\varsigma) \cap E = \{\}$ and $inf(\varsigma) \cap F \neq \{\}$, that is, ς eventually misses all the states of the miss set and infinitely hits some state of the hit set. The run is *accepting* if it is tail-accepting and starts with q_0.

(3) The set of words $L_\omega(\mathcal{R}) = \{w \mid$ some run on w is accepting$\}$ is called the *language accepted by* $\mathcal{R}$.

Given a deterministic RA $\mathcal{R}$, it is easy to build a BA to accept $\Sigma^\omega \setminus L_\omega(\mathcal{R})$, which shows that ω-regular languages are closed under complementation.

THEOREM 2.14. *([Vardi 1996]) Suppose that $\mathcal{B}_1, \mathcal{B}_2$ are Büchi automata with alphabet Σ. Let $L_1 = L_\omega(\mathcal{B}_1)$ and $L_2 = L_\omega(\mathcal{B}_2)$. It is possible to construct Büchi*

automata to accept the following: $L_1 \cap L_2$, $L_1 \cup L_2$, $\Sigma^\omega \setminus L_1$, *and it can be checked algorithmically whether* L_1 *is empty.*

Deciding SCT. There is a decision procedure for SCT based on the fact that the following sets of call-site sequences are ω-regular.

$FLOW = \{cs \mid cs \text{ is infinite and control-flow legal}\}$

$DESC = \{cs_0 cs_1 \mid cs_0 \in Sites^*, \mathcal{M}_{cs_1} \text{ has complete thread with infinite descent}\}$

Consider the BA $\mathcal{B}^{fl} = (Fun \cup \{q_0^{fl}\}, Sites, q_0^{fl}, \Delta^{fl}, Fun)$, where q_0^{fl} is a distinguished start state, and Δ^{fl} contains (q_0^{fl}, c, f), if $c : g \to f$ for some g, and (g, c, f), if $c : g \to f$. It is a simple exercise to show that $\mathcal{B}^{fl}$ accepts *FLOW*.

For *DESC*, build a BA such that an accepting run eventually traces out a thread in some multipath. To allow the run to reflect size changes seen in the thread, a state of the automaton will be a pair consisting of a program parameter and a size change $\gamma \in \{\downarrow, \ddagger\}$. The run will be accepting only if the corresponding thread has infinite descent.

Specifically, define $\mathcal{B}^{de} = (Q^{de}, Sites, q_0^{de}, \Delta^{de}, A^{de})$, where q_0^{de} is a distinguished start state, $Q^{de} = (Par \times \{\downarrow, \ddagger\}) \cup \{q_0^{de}\}$, $A^{de} = Par \times \{\downarrow\}$, and Δ^{de} contains the following transitions:

—(q_0^{de}, c, q_0^{de}), for $c \in Sites$,

—$(q_0^{de}, c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ for some y, and

—$((y, \delta), c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ and $\delta \in \{\downarrow, \ddagger\}$.

It is a simple exercise to show that $\mathcal{B}^{de}$ has an accepting run on cs just when cs can be written as $cs_0 cs_1$, where cs_0 is a finite call-site sequence, and $\mathcal{M}_{cs_1}$ has a complete thread with infinite descent. This shows that the BA accepts *DESC*.

THEOREM 2.15. *It is decidable whether* $\mathcal{G}$ *satisfies SCT.*

PROOF. Construct the automata for *FLOW* and *DESC*. Then construct the BA that accepts $FLOW \setminus DESC = FLOW \cap (Sites^\omega \setminus DESC)$, which is possible by Theorem 2.14. Call it $\mathcal{B}$. Clearly, $\mathcal{B}$ accepts cs just when cs is control-flow legal and $\mathcal{M}_{cs}$ has no infinite descent, that is, just when SCT is violated. It follows that $\mathcal{G}$ is SCT just when $L_\omega(\mathcal{B})$ is empty. By Theorem 2.14, it can be checked algorithmically whether $L_\omega(\mathcal{B})$ is empty. □

2.4 Ranking Functions and SCT

It is now time to connect the satisfaction of SCT by $\mathcal{G}$ with the existence of a $\to_{\mathcal{G}}$-ranking function.

THEOREM 2.16. $\mathcal{G}$ *is SCT* just when $\to_{\mathcal{G}}$ *is well-founded.*

PROOF. (*Soundness*) Suppose that $\mathcal{G}$ is SCT *but* there is an infinite $\to_{\mathcal{G}}$-chain $s_0 \xrightarrow{c_1} s_1 \xrightarrow{c_2} s_2 \ldots$. A contradiction will be derived. First, note that $cs = c_1 c_2 \ldots$ must be control-flow legal, so $\mathcal{M}_{cs}$ is a $\mathcal{G}$-multipath. Since $\mathcal{G}$ satisfies SCT, $\mathcal{M}_{cs}$ has a thread $\vartheta = x_0 \xrightarrow{\gamma_1} x_1 \xrightarrow{\gamma_2} x_2 \ldots \xrightarrow{\gamma_t} x_t \ldots$, where $x_{t-1} \xrightarrow{\gamma_t} x_t \in G_{c_{d+t}}$ for

each $t > 0$, and $\gamma_t = \downarrow$ for infinitely many values of t. Recall that $d \geq 0$ indicates the start position of ϑ in $\mathcal{M}_{cs}$.

For $t \geq 0$, let v_t be the value of x_t in program state s_{d+t}. Then for $t > 0$, it follows from $(s_{d+t-1}, c_{d+t}, s_{d+t}) \in \rightarrow_{\mathcal{G}}$ that $v_t <_D v_{t-1}$ if $\gamma_t = \downarrow$ and $v_t \leq_D v_{t-1}$ if $\gamma_t = \ddagger$. As $\gamma_t = \downarrow$ for infinitely many values of t, the well-foundedness of relation $<_D$ is violated.

(*Completeness*) Suppose that $\mathcal{G}$ is *not* SCT. It will be explained how an infinite $\rightarrow_{\mathcal{G}}$-chain can be defined.

In the proof of Theorem 2.15, it was seen that $FLOW \setminus DESC$ is ω-regular. As $\mathcal{G}$ does not satisfy SCT, there is a control-flow legal call-site sequence whose multipath has no infinite descent. It follows that $FLOW \setminus DESC$ is nonempty. It is well known that a nonempty ω-regular set contains a word of the form $cs = cs_0\, cs_1^{\omega}$, where cs_0, cs_1 are finite words, and cs is the concatenation of cs_0 with infinitely many repetitions of cs_1. Note that $\mathcal{M}_{cs}$ is a multipath without infinite descent.

An infinite $\rightarrow_{\mathcal{G}}$-chain can be defined by assigning D values to the nodes of $\mathcal{M}_{cs}$ in a way that respects the size-change arcs of $\mathcal{M}_{cs}$. Instead of directly assigning values of D, natural numbers can be used. A number n will then be interpreted as $(n)_D$, the n-th smallest element of D. An assignment to the nodes of a multipath that respects the arcs of the multipath will be called *consistent*.

Let $\mathcal{M}_{cs} = f_0 \overset{G_{c_1}}{\rightarrow} f_1 \overset{G_{c_2}}{\rightarrow} f_2 \ldots$. As a digraph, the nodes of this multipath consists of infinitely many partitions, with the nodes of partition t labeled by $Param(f_t)$, for $t \geq 0$. Let the length of cs_0 be n_0 and suppose that numbers have been assigned consistently to the nodes of $\mathcal{M}_{(cs_1)^{\omega}}$. Then the same numbers can be assigned to the nodes of $\mathcal{M}_{cs}$ starting from partition n_0. Let the maximum number assigned to the nodes of partition n_0 be m. For $0 \leq t < n_0$, assigning all the nodes of partition t the number $m + (n_0 - t)$ is consistent and completes the job.

Can numbers be assigned consistently to the nodes of $\mathcal{M}_{cs_1^{\omega}}$? Note that the multipath $\mathcal{M}_{cs_1}$ has to start and end at the same program function. Consider the digraph $\mathcal{M}^{\circ}_{cs_1}$, obtained by identifying each node at the end of $\mathcal{M}_{cs_1}$ with the node labeled by the same parameter at the start of the multipath, that is, by directing the arcs between the final two partitions of $\mathcal{M}_{cs_1}$ to nodes at the start of the multipath. Clearly, any consistent assignment to the nodes of $\mathcal{M}^{\circ}_{cs_1}$ implies a consistent assignment to the nodes of $\mathcal{M}_{cs_1^{\omega}}$.

Now, $\mathcal{M}^{\circ}_{cs_1}$ cannot contain any cycle with a strict arc, for otherwise $\mathcal{M}_{cs_1^{\omega}}$ would have a thread with infinite descent. This means that the nodes in each strongly connected component of $\mathcal{M}^{\circ}_{cs_1}$ are connected to one another by nonstrict arcs. It is possible to arrange the SCCs of $\mathcal{M}^{\circ}_{cs_1}$ in reverse topological order [Cormen et al. 2001]. Let $C_1, \ldots, C_N$ be such an ordering. Then any strict arc from a node of C_i to a node of C_j implies that $j < i$. For each node in C_i, assign it the number i. This assignment is consistent. □

COROLLARY 2.17. *Let $\mathcal{G}$ be a set of size-change graphs that is safe for $\rightarrow_p$, the transition relation of the subject program p. If $\mathcal{G}$ is SCT, then p is terminating.*

PROOF. Suppose that $\mathcal{G}$ satisfies SCT. By Theorem 2.16, $\rightarrow_{\mathcal{G}}$ is well-founded. By assumption, $\mathcal{G}$ is safe for $\rightarrow_p$, so $\rightarrow_p$ is a subset of $\rightarrow_{\mathcal{G}}$. It follows that $\rightarrow_p$ is well-founded. Finally, by Corollary 2.6, subject program p is terminating. □

The strategy of size-change analysis is now plain. From the subject program p, derive a set of size-change graphs $\mathcal{G}$ whose transition relation over-approximates the transition relation of p, then test the well-foundedness of the larger transition relation by checking SCT. If the test succeeds, termination of p is verified.

Specifically, for Example 1.1 (Ackermann), the accompanying size-change graphs $\mathcal{G}$ are first derived by inspecting the program's function calls. Semantically, the program's transition relation is a subset of $\rightarrow_{\mathcal{G}}$, which corresponds to a nondeterministic loop where either `m` is decreased and `n` is set arbitrarily, or `m` is not increased and `n` is decreased. This loop is terminating just when $\mathcal{G}$ is SCT. It can be verified mechanically that $\mathcal{G}$ is, in fact, SCT. It follows that $\rightarrow_{\mathcal{G}}$ is well-founded. Consequently, the program's transition relation is well-founded, and the program is terminating.

There is another test of the SCT property that does not involve ω-automata [Lee et al. 2001]. This test is possibly more suited for implementation [Frederiksen 2001; Wahlstedt 2000; Thiemann and Giesl 2003; Jones and Bohr 2004]. However, it is the ω-automaton approach that is relevant to the result in this article.

The problem, defined. Let $\mathcal{G}$ be a given set of size-change graphs. For succinctness, refer to a $\rightarrow_{\mathcal{G}}$-ranking function as a $\mathcal{G}$-ranking function.

The problem tackled in this article can now be stated as follows: Given that $\mathcal{G}$ satisfies SCT, is there a way to construct a $\mathcal{G}$-ranking function? If $\mathcal{G}$ is safe for $\rightarrow_p$, the transition relation of the subject program p, the constructed ranking function is automatically a $\rightarrow_p$-ranking function.

3. RESTRICTED PROBLEMS

As a stepping-stone to the main result of this article—a procedure for constructing $\mathcal{G}$-ranking functions—a very simple restricted problem will first be considered. This is followed by a review of a BA determinization procedure [Althoff et al. 2005] needed in the correctness proof of a more complex ranking-function construction.

3.1 Strict Arcs Only

Throughout this subsection, it will be assumed that $\mathcal{G}$ is a set of size-change graphs with the following properties:

(1) There is only one program function, denoted f. For any call-site c, $G_c : f \rightarrow f$.
(2) The size-change graphs of $\mathcal{G}$ have only strict arcs.
(3) $\mathcal{G}$ satisfies SCT.

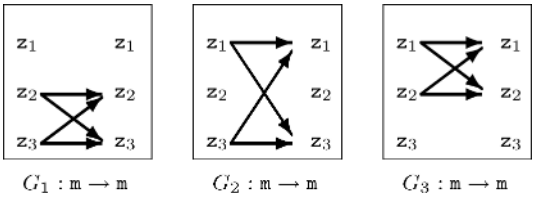


Fig. 3. Mystery descent 2005, with strict arcs only.

Example 3.1. (Mystery descent 2005)
Such a restricted $\mathcal{G}$, due to Amir Ben-Amram, is depicted in Figure 3.
Now, consider any infinite $\mathcal{G}$-multipath:

$$\mathcal{M} = \mathtt{m} \overset{G_{c_1}}{\to} \mathtt{m} \overset{G_{c_2}}{\to} \mathtt{m} \ldots,$$

and define a parameter sequence as follows. Pick $x_0 = \mathtt{z}_i$ where $i \in \{1, 2, 3\} \setminus \{c_1\}$, and for each $t > 0$, pick $x_t = \mathtt{z}_i$ where $i \in \{1, 2, 3\} \setminus \{c_t, c_{t+1}\}$. It is a simple matter to verify that $x_0 \overset{\downarrow}{\to} x_1 \overset{\downarrow}{\to} x_2 \ldots$ is a complete thread in $\mathcal{M}$. Consequently, $\mathcal{M}$ has infinite descent, and $\mathcal{G}$ satisfies SCT.

So what function of the parameter values decreases for every state transition of $\to_{\mathcal{G}}$? Ben-Amram made the fascinating observation that the *median* of the parameter values always decreases. A more general development follows.

Definition 3.2. Let $c \in Sites$.

(1) For $x \in Par$, let $down_c(x) = \{x' \mid x \overset{\downarrow}{\to} x' \in G_c\}$.
(2) For $X \subseteq Par$, let $down_c(X) = \bigcup_{x \in X} down_c(x)$.

By the definition of $down_c$, if $x' \in down_c(x)$ and $s \overset{c}{\to}_{\mathcal{G}} s'$, then the value of x' in s' is strictly less than the value of x in s. What is the "value" of a set X in s?

Definition 3.3. For $s \in St$ and $X \subseteq Par$, let:

$$\text{MAXVAL}_X(s) = \max\{v \mid x \in X,\ v \text{ is the value of } x \text{ in } s\}.$$

LEMMA 3.4. *Suppose that $X' \subseteq down_c(X)$, X' is non-empty, and $s \overset{c}{\to}_{\mathcal{G}} s'$. Then* $\text{MAXVAL}_{X'}(s') <_D \text{MAXVAL}_X(s)$.

PROOF. It follows from $s \overset{c}{\to}_{\mathcal{G}} s'$ and the definition of $down_c$ that for any $x' \in X'$, the value of x' in s' is strictly less than the value of some $x \in X$ in s, hence strictly less than $\text{MAXVAL}_X(s)$. In particular, as X' is not empty, $\text{MAXVAL}_{X'}(s')$ is the value of some $x' \in X'$ in s'. Therefore, $\text{MAXVAL}_{X'}(s') <_D \text{MAXVAL}_X(s)$. □

LEMMA 3.5. *Suppose that $\mathcal{S} \subseteq \wp(Par)$ is nonempty, and for every $X \in \mathcal{S}$ and $c \in Sites$, there is some nonempty $X' \in \mathcal{S}$ such that $X' \subseteq down_c(X)$. Then $\rho_f(s) = \min\{\text{MAXVAL}_X(s) \mid X \in \mathcal{S}\}$ is a $\mathcal{G}$-ranking function.*

PROOF. Let $s \overset{c}{\to}_{\mathcal{G}} s'$. By assumption, for any $X \in \mathcal{S}$, there is some nonempty $X' \in \mathcal{S}$ such that $X' \subseteq down_c(X)$. By Lemma 3.4, $\text{MAXVAL}_{X'}(s') <_D \text{MAXVAL}_X(s)$. Clearly, the value of $\rho_f(s')$ is no greater than $\text{MAXVAL}_{X'}(s')$. It follows that for

any $X \in \mathcal{S}$, $\rho_f(s') <_D \textsc{maxval}_X(s)$. As $\mathcal{S}$ is nonempty, there is in fact some X for which $\rho_f(s) = \textsc{maxval}_X(s)$. Therefore, $\rho_f(s') <_D \rho_f(s)$. □

LEMMA 3.6. *Let $\mathcal{S}$ be the least set that satisfies the following.*

—$Par \in \mathcal{S}$, and

—if $c \in Sites$ and $X \in \mathcal{S}$, then $down_c(X) \in \mathcal{S}$.

Then the empty set is not an element of $\mathcal{S}$.

PROOF. For any $X \in \mathcal{S}$, there exists a sequence $X_0, X_1, \ldots, X_T$ of subsets of Par and $c_1, \ldots, c_T \in Sites$ such that $X_0 = Par$, $X_T = X$, and $down_{c_t}(X_{t-1}) = X_t$ for $0 < t \leq T$. If $T = 0$, then $X = Par \neq \{\}$, so suppose that $T > 0$, and let $cs = c_1 \ldots c_T$.

The multipath $\mathcal{M}_{cs}$ contains a complete thread, for otherwise $\mathcal{M}_{cs^\omega}$ would have no infinite thread and $\mathcal{G}$ would not be SCT. Let the thread be $x_0 \overset{\downarrow}{\rightarrow} x_1 \ldots \overset{\downarrow}{\rightarrow} x_T$. By induction, $x_T \in X_T = X$. Therefore, $X \neq \{\}$. □

COROLLARY 3.7. *Let $\rho_f(s) = \min\{\textsc{maxval}_X(s) \mid X \in \mathcal{S}\}$, where $\mathcal{S}$ is defined as in Lemma 3.6. Then ρ_f is a $\mathcal{G}$-ranking function.*

PROOF. First, observe that $\mathcal{S}$ is not empty since it contains Par as an element. By Lemma 3.6, for every $X \in \mathcal{S}$ and $c \in Sites$, there is some nonempty $X' \in \mathcal{S}$ such that $X' = down_c(X)$. By Lemma 3.5, the ρ_f stated is a $\mathcal{G}$-ranking function. □

The definition of the set $\mathcal{S}$ in Lemma 3.6 resembles the subset construction for determinizing finite state automata. This observation turns out to be central for the unrestricted problem.

Lemma 3.5 can be used to verify that a given subset of $\wp(Par)$ induces a $\mathcal{G}$-ranking function, since the conditions of the lemma are easy to check. Returning to Example 3.1, the set $\mathcal{S} = \{\{z_2, z_3\}, \{z_1, z_3\}, \{z_1, z_2\}\}$ satisfies these conditions. It follows that ρ_m below is a $\mathcal{G}$-ranking function for the example.

$$\rho_m(z_1, z_2, z_3) = \min\{\max\{z_2, z_3\}, \max\{z_1, z_3\}, \max\{z_1, z_2\}\}.$$

Note that $\rho_m(z_1, z_2, z_3)$ is precisely the median of z_1, z_2, and z_3, validating an earlier claim that this value decreases in every state transition of $\rightarrow_{\mathcal{G}}$.

For the restricted type of $\mathcal{G}$ considered in this subsection, a brute-force search for a $\mathcal{G}$-ranking function can be carried out as follows: Enumerate the subsets of $\wp(Par)$ and check whether any satisfies the conditions of Lemma 3.5. A $\mathcal{G}$-ranking function is found this way just when $\rightarrow_{\mathcal{G}}$ is well-founded.

3.2 Single-Function $\mathcal{G}$

In this section, the size-change graphs of $\mathcal{G}$ are allowed to have nonstrict arcs, but the other assumptions remain in place: There is only one program function, denoted f, and $\mathcal{G}$ satisfies SCT.

The construction of a $\mathcal{G}$-ranking function is analogous to the preceding section, but requires a revision of $down_c$. Instead of operating on parameters or

sets of parameters, it now operates on tuples of parameters and constants, or sets of such tuples. In the remainder of this article, n and k will be used consistently to refer to the number of constants allowed in a tuple and the length of a tuple respectively. The reader is referred to the ranking function given for Example 1.3 for the form of such tuples.

If $\vec{z}' \in down_c(\vec{z})$ and $s \xrightarrow{c}_{\mathcal{G}} s'$, then the "value" of $\vec{z}'$ in s' should be strictly less than the "value" of $\vec{z}$ in s. This is formalized as follows.

Definition 3.8. Let $c \in Sites$ and n, k be positive integers.

(1) Let $Par_n = Par \cup \{0, \ldots, n-1\}$. Then Par_n^k is the set of k-tuples whose elements are parameters or numbers between 0 and $n-1$.
(2) For $\vec{z} = \langle z_1, \ldots, z_k \rangle, \vec{z}' = \langle z'_1, \ldots, z'_k \rangle \in Par_n^k$, write $\vec{z}' \prec_c \vec{z}$ if for some i,
    —either $z_i, z'_i \in Par$ and $z_i \xrightarrow{\downarrow} z'_i \in G_c$, or $z_i, z'_i \in \mathbb{N}$ and $z'_i = z_i - 1$; and
    —for each $j < i$, either $z_j, z'_j \in Par$ and $z_j \xrightarrow{\gamma} z'_j \in G_c$,
    or $z_j, z'_j \in \mathbb{N}$ and $z'_j = z_j$.
(3) For $\vec{z} \in Par_n^k$, let $down_c(\vec{z}) = \{\vec{z}' \mid \vec{z}' \prec_c \vec{z}\}$.
(4) For $Z \subseteq Par_n^k$, let $down_c(Z) = \bigcup_{\vec{z} \in Z} down_c(\vec{z})$.

Definition 3.9. (Valuation of tuples and sets of tuples)

(1) Let $s \in St$. For $\vec{z} = \langle z_1, \ldots, z_k \rangle \in Par_n^k$, the valuation of $\vec{z}$ in s is the k-tuple $\langle v_1, \ldots, v_k \rangle$, where for $1 \le i \le k$, if $z_i \in Par$, then v_i is the value of z_i in s, and if z_i is a number, then $v_i = (z_i)_D$. Recall that $(z)_D$ is the z-th smallest element of D under $\le_D$.
(2) The set D^k is well-ordered under the lexicographic extension of $\le_D$. Denote this relation on D^k by $\le_{D^k}$, and its strict part by $<_{D^k}$. When clear from context, the subscript D^k is dropped. The maximum over a finite subset of D^k is taken with respect to this well-ordering. For $Z \subseteq Par_n^k$, define $\textsc{maxval}_Z : St \to D^k$ as:
$$\textsc{maxval}_Z(s) = \max\{\vec{v} \mid \vec{z} \in Z,\ \vec{v} \text{ is the valuation of } \vec{z} \text{ in } s\}.$$

Lemma 3.10. *Let $\vec{z} = \langle z_1, \ldots, z_k \rangle$ and $\vec{z}' = \langle z'_1, \ldots, z'_k \rangle$ be elements of Par_n^k with $\vec{z}' \prec_c \vec{z}$. Suppose that $s \xrightarrow{c}_{\mathcal{G}} s'$. Let $\vec{v} = \langle v_1, \ldots, v_k \rangle$ be the valuation of $\vec{z}$ in s and $\vec{v}' = \langle v'_1, \ldots, v'_k \rangle$ the valuation of $\vec{z}'$ in s'. Then $\vec{v}' < \vec{v}$.*

Proof. By the definition of $\prec_c$, there is some i such that either $z_i, z'_i \in Par$ and $z_i \xrightarrow{\downarrow} z'_i \in G_c$, or $z_i, z'_i \in \mathbb{N}$ and $z'_i = z_i - 1$. In the first case, it follows from $s \xrightarrow{c}_{\mathcal{G}} s'$ and the definition of the valuation of tuples that $v'_i <_D v_i$, while in the second case, $v'_i = (z'_i)_D <_D (z_i)_D = v_i$.

The definition of $\prec_c$ also requires that for $j < i$, one of the following holds: $z_j, z'_j \in Par$ and $z_j \xrightarrow{\gamma} z'_j \in G_c$, or $z_j, z'_j \in \mathbb{N}$ and $z'_j = z_j$. Argue as above that $v'_j \le_D v_j$. By the definition of lexicographic ordering, $\vec{v}' < \vec{v}$ in D^k. □

Lemma 3.11. *Suppose that $Z, Z' \subseteq Par_n^k$, $Z' \subseteq down_c(Z)$, Z' is nonempty, and $s \xrightarrow{c}_{\mathcal{G}} s'$. Then $\textsc{maxval}_{Z'}(s') < \textsc{maxval}_Z(s)$.*

PROOF. By the definition of $down_c$, for any $\vec{z}' \in Z'$, there is some $\vec{z} \in Z$ such that $\vec{z}' \prec_c \vec{z}$. By Lemma 3.10, the valuation of $\vec{z}'$ in s' is strictly less than the valuation of $\vec{z}$ in s, hence strictly less than $\text{MAXVAL}_Z(s)$. In particular, as Z' is non-empty, $\text{MAXVAL}_{Z'}(s')$ is the valuation of some $z' \in Z'$ in s'. Therefore, $\text{MAXVAL}_{Z'}(s') < \text{MAXVAL}_Z(s)$. □

LEMMA 3.12. *Suppose that $\mathcal{S} \subseteq \wp(Par_n^k)$ is nonempty, and for every $Z \in \mathcal{S}$ and $c \in Sites$, there is a non-empty $Z' \in \mathcal{S}$ such that $Z' \subseteq down_c(Z)$. Then $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid Z \in \mathcal{S}\}$ is a $\mathcal{G}$-ranking function.*

PROOF. Let $s \overset{c}{\rightarrow}_{\mathcal{G}} s'$. By assumption, for any $Z \in \mathcal{S}$, there is some nonempty $Z' \in \mathcal{S}$ such that $Z' \subseteq down_c(Z)$. By Lemma 3.11, $\text{MAXVAL}_{Z'}(s') < \text{MAXVAL}_Z(s)$. Clearly, the value of $\rho_f(s')$ is no greater than $\text{MAXVAL}_{Z'}(s')$. It follows that for any $Z \in \mathcal{S}$, $\rho_f(s') < \text{MAXVAL}_Z(s)$. As $\mathcal{S}$ is nonempty, there is in fact some Z for which $\rho_f(s) = \text{MAXVAL}_Z(s)$. Based on this instance of Z, $\rho_f(s') < \rho_f(s)$. □

LEMMA 3.13. *Let $\mathcal{S}_{n,k}$ be the least set that satisfies the following:*

—$Par_n^k \in \mathcal{S}_{n,k}$, and
—if $c \in Sites$ and $Z \in \mathcal{S}_{n,k}$, then $down_c(Z) \in \mathcal{S}_{n,k}$.

Suppose it holds that $\{\} \notin \mathcal{S}_{n,k}$. Then $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid Z \in \mathcal{S}_{n,k}\}$ is a $\mathcal{G}$-ranking function.

PROOF. The set $\mathcal{S}_{n,k}$ is not empty since it contains Par_n^k as an element. The next claim follows from the definition of $\mathcal{S}_{n,k}$ and the assumption that $\{\} \notin \mathcal{S}_{n,k}$: For every $Z \in \mathcal{S}_{n,k}$ and $c \in Sites$, there is some nonempty $Z' \in \mathcal{S}_{n,k}$ such that $Z' = down_c(Z)$. By Lemma 3.12, the ρ_f stated is a $\mathcal{G}$-ranking function. □

It therefore remains to show that for suitable values of n and k, the closure $\mathcal{S}_{n,k}$ does not contain the empty set. Based on these values of n and k, the closure can then be computed and a ranking function derived by Lemma 3.13.

Before embarking on a proof that these values exist, observe how Lemma 3.12 can be used to verify that a given subset of $\wp(Par_n^k)$ other than $\mathcal{S}_{n,k}$ induces a $\mathcal{G}$-ranking function, since the lemma's conditions are easy to check. Consider Example 1.8. The ranking function given was: $\rho_{\mathtt{m}}(x, y, z, w) = \max\{\langle x, z\rangle, \langle y, w\rangle\}$.

It can be confirmed as a $\mathcal{G}$-ranking function for the example by checking that $\mathcal{S} = \{\{\langle \mathtt{x}, \mathtt{z}\rangle, \langle \mathtt{y}, \mathtt{w}\rangle\}\}$ satisfies the conditions of Lemma 3.12. In this case, it only has to be checked that for $c = 1, 2$ and $Z = \{\langle \mathtt{x}, \mathtt{z}\rangle, \langle \mathtt{y}, \mathtt{w}\rangle\}$, $Z \subseteq down_c(Z)$. This holds because for each c and $\vec{z}' \in Z$, there is some $\vec{z} \in Z$ such that $\vec{z}' \prec_c \vec{z}$. Specifically, for $c = 1$, $\langle \mathtt{x}, \mathtt{z}\rangle \prec_1 \langle \mathtt{x}, \mathtt{z}\rangle$ because $\mathtt{x} \overset{\mp}{\rightarrow} \mathtt{x}$, $\mathtt{z} \overset{\downarrow}{\rightarrow} \mathtt{z} \in G_1$, and $\langle \mathtt{y}, \mathtt{w}\rangle \prec_1 \langle \mathtt{x}, \mathtt{z}\rangle$ because $\mathtt{x} \overset{\downarrow}{\rightarrow} \mathtt{y} \in G_1$. The case of $c = 2$ is similar.

It is not hard to see that if $\mathcal{S}_{n,k}$ does not contain the empty set, then neither does $\mathcal{S}_{n',k'}$ for $n' \geq n$ and $k' \geq k$. Therefore, given upper bounds n', k' for suitable values of n and k, a brute-force search for a $\mathcal{G}$-ranking function can proceed as follows: Enumerate the subsets of $\wp(Par_{n'}^{k'})$ and check whether any satisfies the conditions of Lemma 3.12. In principle, a $\mathcal{G}$-ranking function will be found this way just when $\rightarrow_{\mathcal{G}}$ is well-founded. Clearly, such a procedure would not be practical.

For theoretical interest then, do there always exist n and k such that $\mathcal{S}_{n,k}$ does not contain the empty set?

Definition 3.14. Let $cs = c_1c_2 \ldots c_T$ be any finite call-site sequence. A sequence of Par_n^k elements $\vec{z}^{(0)}, \vec{z}^{(1)}, \ldots, \vec{z}^{(T)}$ is an (n,k)-*progression on* cs if $\vec{z}^{(t)} \prec_{c_t} \vec{z}^{(t-1)}$ for $0 < t \leq T$.

LEMMA 3.15. *Assume there exists an* (n,k)*-progression on any finite call-site sequence. Then* $\mathcal{S}_{n,k}$ *does not contain the empty set.*

PROOF. For any $Z \in \mathrm{S}_{n,k}$, there exists a sequence $Z_0, Z_1, \ldots, Z_T$ of subsets of Par_n^k and $c_1, \ldots, c_T \in Sites$ such that $Z_0 = Par_n^k$, $Z_T = Z$, and $down_{c_t}(Z_{t-1}) = Z_t$ for $0 < t \leq T$. If $T = 0$, then $Z = Par_n^k \neq \{\}$, so suppose that $T > 0$, and let $cs = c_1 \ldots c_T$.

By assumption, there exist $\vec{z}^{(0)}, \vec{z}^{(1)}, \ldots, \vec{z}^{(T)} \in Par_n^k$ satisfying $\vec{z}^{(t)} \prec_{c_t} \vec{z}^{(t-1)}$ for $0 < t \leq T$. By induction, $\vec{z}^{(T)} \in Z_T$. Therefore, $Z \neq \{\}$. □

The fact that for suitable values of n and k, there exists an (n,k)-progression on *any* finite call-site sequence will be teased from the determinization of $\mathcal{B}^{de}$. Recall that $\mathcal{B}^{de}$ was defined earlier for the proof of Theorem 2.15.

3.3 Determinization of $\mathcal{B}^{de}$

The BA determinization procedure below is the work of Muller and Schupp [1995], but the presentation follows the conventions of a recent exposition by Althoff et al. [2005]. For readers familiar with the subset construction for determinizing finite state automata, the Muller-Schupp procedure can be seen as an extension of this procedure, designed to cope with a semantically more complicated acceptance condition.

Connection with the problem of defining a progression on cs. Before we delve into the intricacies of the Muller-Schupp procedure, what is the role here of the deterministic automaton $\mathcal{R}^{de}$ derived from $\mathcal{B}^{de}$? Clearly, it is not for manipulating the language accepted: Satisfaction of SCT by $\mathcal{G}$ readily implies that $\mathcal{B}^{de}$ and $\mathcal{R}^{de}$ are universal, that is, they accept every word. The actual reason for considering $\mathcal{R}^{de}$ is because there is a natural way to define an (n,k)-progression on any labeling cs of a given $\mathcal{R}^{de}$-run ς using bounded values of n and k.

This definition is inductive in the number of distinct states appearing in ς. It is presented informally here in three cases, with each case reducing to the next, and the final case reducing to a smaller problem. The actual development will deviate from this outline, combining Cases 1 and 3 to obtain a tighter bound for a suitable k. Let m denote the number of states in $\mathcal{R}^{de}$.

Case 1: Suppose that the restriction of $\mathcal{R}^{de}$ to states in ς has multiple strongly connected components. By associating with each point in ς a number based on the component of the state appearing at that point, it is straightforward to define a sequence of natural numbers, all less than m, that decrease upon each intercomponent transition. By making this sequence the top position of the progression being defined, the problem reduces to the following case.

Case 2: Suppose that states in ς are strongly connected in $\mathcal{R}^{de}$. As with the usual subset construction, a set of $\mathcal{B}^{de}$-runs on cs can be extracted from ς. Each run corresponds to a complete thread in the multipath $\mathcal{M}_{cs}$. A consequence of the universality of $\mathcal{R}^{de}$ is the following: There is a complete thread ϑ of $\mathcal{M}_{cs}$ and a nonempty subset ς of the states of ς such that if ς has two occurrences of an element of ς, then the segment of ϑ corresponding to these occurrences has a decreasing arc. By making the parameters of ϑ the top position of the progression being defined, the problem reduces to the next case.

Case 3: Suppose that states in ς are strongly connected in $\mathcal{R}^{de}$ and a nonempty subset ς of these states occur in ς at most once. A nonascending sequence of natural numbers with a starting value less than $2m$ can be associated with each point in ς so that decreases are observed before and after every occurrenc of a state of ς. By making this sequence the top position of the progression being defined, the problem reduces to one where the $\mathcal{R}^{de}$-run has fewer distinct states.

Suitable values for n and k (independent of ς) will depend, respectively, on the natural numbers needed in the actual definition and the inductive depth of this definition.

The Muller-Schupp procedure. In the following, let $\mathcal{B} = (Q, \Sigma, q_0, \Delta_Q, A_Q)$ be the BA to be determinized. It is assumed that $q_0 \notin A_Q$.

The Muller-Schupp procedure manipulates a binary-tree structure dubbed the Muller-Schupp tree in Althoff et al. [2005]. This is used as an abstraction for a set of $\mathcal{B}$-runs starting from q_0. Each leaf of the tree is labeled by a subset of Q, called a macrostate. When making a transition, each macrostate is updated via the usual subset construction, and the result Q' is split into $Q' \cap A_Q$ and $Q' \setminus A_Q$. These sets are made the macrostates of new leaf nodes, which are attached to the current leaf. A *cancellation policy* trims the resulting structure, keeping the space of Muller-Schupp trees finite, while maintaining essential information regarding occurrences of accepting states.

Definition 3.16. A *Muller-Schupp tree* τ is a finite sibling-ordered full binary-tree, that is, a finite tree in which every nonleaf node has exactly two children—a left child and a right child. Every node of the tree is labeled with the following:

—A positive integer label λ, called the node ID;

—A color from $\{\mathbf{R}, \mathbf{Y}, \mathbf{G}\}$, standing for Red, Yellow, and Green;

—A subset of Q, called the macrostate.

Different nodes have distinct IDs in τ. A node is therefore uniquely identified by its ID. For succinctness, node λ refers to the node with ID λ. Use $mstate_\lambda(\tau)$ to denote the macrostate of node λ in τ. The macrostates of leaf nodes are mutually disjoint, and the macrostate of a non-leaf node is the union of its children's macrostates. The tree τ is *normalized* if every macrostate is nonempty.

As observed in Althoff et al. [2005], the macrostates of nonleaf nodes are redundant, as they can be recovered from macrostates at the leaves. They do, however, make the determinization procedure easier to explain.

Procedure $update_a(\tau)$, where τ is a normalized Muller-Schupp tree.

(1) Let Λ be an ascending enumeration of the positive integers, *excluding* those used as IDs in τ. New IDs will be drawn from Λ in order.
(2) Make a copy τ' of τ. The operations below are performed successively on τ' and the final result is returned by the procedure.
(3) Reassign color $\mathbf{Y}$ to any node in τ' with color $\mathbf{G}$.
(4) For each leaf node λ, let $mstate_\lambda(\tau') = \{r' \mid r \in mstate_\lambda(\tau),\ (r, a, r') \in \Delta_Q\}$.
(5) If every macrostate assigned in Step 4 is empty, return ε.
(6) In a depth-first traversal of τ', process each leaf node λ as follows.
    (a) Let $Q_{left} = mstate_\lambda(\tau') \cap A_Q$ and $Q_{right} = mstate_\lambda(\tau') \setminus A_Q$.
    (b) Let λ_{left} and λ_{right} be the next labels from Λ. Attach a node with ID λ_{left}, color $\mathbf{G}$, and macrostate Q_{left} as the left child of λ. Attach a node with ID λ_{right}, color $\mathbf{R}$, and macrostate Q_{right} as the right child of λ.
(7) Keep only the left-most occurrence of each $\mathcal{B}$-state seen in the leaves of τ', as follows. Initialize Q_{seen} to the empty set. Then apply the following steps to each leaf λ encountered in a depth-first traversal of τ'.
    (a) Let $mstate_\lambda(\tau') = mstate_\lambda(\tau') \setminus Q_{seen}$.
    (b) Let $Q_{seen} = Q_{seen} \cup mstate_\lambda(\tau')$.
(8) Set the macrostates of all non-leaf nodes to $\{\}$. Then repeatedly set a non-leaf node's macrostate to the union of its children's until every one is stable.
(9) Normalization: Delete every node with an empty macrostate, and repeatedly apply the following steps to any node λ with an only child ν.
    (a) If ν has color $\mathbf{G}$ or $\mathbf{Y}$, reassign color $\mathbf{G}$ to λ.
    (b) Attach any children of ν directly to λ.
    (c) Delete ν.
(10) Return the normalized Muller-Schupp tree τ'.

Fig. 4. Update of a normalized Muller-Schupp tree.

Some terminology about trees: In a tree τ, node λ is *above* node ν if $\lambda = \nu$ or λ is above the parent of ν; λ is *strictly* above ν if λ is above ν but $\lambda \neq \nu$; ν is (strictly) *below* λ if λ is (strictly) above ν.

Building an RA. Muller-Schupp trees will be used as *states* of the deterministic RA constructed. Let $a \in \Sigma$. The reader should take a moment to study the deterministic procedure $update_a$ of Figure 4, which takes a Muller-Schupp tree and returns an *updated* one. It has been written for clarity rather than efficiency.

The *initial tree* τ_* contains a single node labeled by integer 1, color $\mathbf{R}$, and macrostate $\{q_0\}$. A deterministic Rabin automaton $\mathcal{R}$ is constructed as follows. First, the states of $\mathcal{R}$ is the least set R such that:

—$\tau_* \in R$, and
—if $\tau \in R$, $a \in \Sigma$, and $\tau' = update_a(\tau) \neq \varepsilon$, then $\tau' \in R$.

By the definition of *update*, R contains only normalized Muller-Schupp trees. Since the number of such trees is finite, R can be computed in a straightforward way. The transition relation Δ_R contains (τ, a, τ') if $update_a(\tau) = \tau'$. Every ID λ seen in R leads to a Rabin acceptance pair (E_λ, F_λ), where E_λ contains τ if τ has *no* node with ID λ, and F_λ contains τ if τ has a node with ID λ and color $\mathbf{G}$. Refer to (E_λ, F_λ) as the *acceptance pair indexed by* λ. The acceptance condition is given by the set Ω_R of all acceptance pairs. Thus, an $\mathcal{R}$-run ς is accepting if

and only if some node λ appears in every tree after some point in ς, and this node has color **G** infinitely often in ς.

THEOREM 3.17. *([Muller and Schupp 1995; Althoff et al. 2005]) The deterministic RA given by $\mathcal{R} = (R, \Sigma, \tau_*, \Delta_R, \Omega_R)$ accepts the same language as $\mathcal{B}$.*

There is a lemma supporting Theorem 3.17 that is critical to the main result in this article. It will be proved as Lemma 3.24.

Definition 3.18. (Node chains)

(1) Let λ be a node of τ and λ' a node of τ'. For $a \in \Sigma$, write $(\tau : \lambda) \trianglerighteq_a (\tau' : \lambda')$ if $mstate_{\lambda'}(\tau') \subseteq \{r' \mid r \in mstate_\lambda(\tau),\ (r, a, r') \in \Delta_Q\}$.

(2) Let $\varsigma = \tau_0\tau_1 \ldots \tau_T$ be an $\mathcal{R}$-run labeled by $w = a_1 \ldots a_T$. A *node chain* over ς, w is a sequence of node IDs $\lambda_0\lambda_1 \ldots \lambda_T$ such that each λ_t is a node of τ_t, and for $0 < t \leq T$, $(\tau_{t-1} : \lambda_{t-1}) \trianglerighteq_{a_t} (\tau_t : \lambda_t)$.

In other words, $(\tau : \lambda) \trianglerighteq_a (\tau' : \lambda')$ means that the macrostate at node λ' of τ' is a subset of the result of applying the subset construction to the macrostate at node λ of τ. Consequently, a node chain over ς, w yields a $\mathcal{B}$-run on w.

LEMMA 3.19. *Let $\varsigma = \tau_0\tau_1 \ldots \tau_T$ be an $\mathcal{R}$-run on $w = a_1 \ldots a_T$. For any node chain $\lambda_0\lambda_1 \ldots \lambda_T$ over ς, w and $r \in mstate_{\lambda_T}(\tau_T)$, there exists some $\mathcal{B}$-run $\varrho = r_0 r_1 \ldots r_T$ on w such that each $r_t \in mstate_{\lambda_t}(\tau_t)$ and $r_T = r$.*

PROOF. Perform induction on the length of ς. The induction hypothesis is that the lemma holds if this length is smaller than T. By the definition of a node chain, $(\tau_{T-1} : \lambda_{T-1}) \trianglerighteq_{a_T} (\tau_T : \lambda_T)$. By the definition of $\trianglerighteq$, for $r \in mstate_{\lambda_T}(\tau_T)$, there exists $q \in mstate_{\lambda_{T-1}}(\tau_{T-1})$ such that $(q, a_T, r) \in \Delta_Q$. If $T = 1$, then qr is the required $\mathcal{B}$-run on w. If $T > 1$, it follows from the induction hypothesis that there exists a $\mathcal{B}$-run $r_0 r_1 \ldots r_{T-1}$ on $a_1 \ldots a_{T-1}$ such that $r_t \in mstate_{\lambda_t}(\tau_t)$ for each t, and $r_{T-1} = q$. In this case, the required $\mathcal{B}$-run on w is $r_0 \ldots r_{T-1} r$. □

The next observations follow easily from the definition of *update*.

LEMMA 3.20. *Let $(\tau, a, \tau') \in \Delta_R$.*

(1) If τ and τ' each has a node with ID λ, then $(\tau : \lambda) \trianglerighteq_a (\tau' : \lambda)$.

(2) Let λ' be a node of τ'. If τ does not *have a node with ID λ', then it has a node λ such that λ is the parent of λ' in τ', and $(\tau : \lambda) \trianglerighteq_a (\tau' : \lambda')$.*

In fact, the macrostate at the root of τ' is precisely the result of applying the subset construction to the macrostate at the root of τ. In this sense, the Muller-Schupp procedure is an extension of the usual subset construction. This extension is necessary to track occurrences of $\mathcal{B}$-accepting states properly.

Definition 3.21. Let $(\tau, a, \tau') \in \Delta_R$. Suppose that τ and τ' each has a node with ID λ. If ν occurs in τ strictly below λ, and any node above ν (including ν) and strictly below λ has an ID that does *not* appear in τ', say that ν is *contracted into* λ on update of τ by a.

LEMMA 3.22. *Let $(\tau, a, \tau') \in \Delta_R$. If τ has a node ν with color **G** or **Y** that is contracted into λ on update by a, then τ' has a node λ with color **G**.*

PROOF. Let Λ be the set of IDs above ν and below λ in τ. For convenience, refer to a node with an ID in Λ as a Λ-node.

Each member of $\Lambda \setminus \{\lambda\}$ is the ID of a node deleted during Step 9 of $update_a(\tau)$ (the normalization step). At some point, node ν is deleted, causing some undeleted Λ-node to be assigned color **G**. It is easy to verify that any subsequent deletion of a Λ-node leaves behind some undeleted Λ-node with color **G**. Therefore, in τ', which is the result of the update, node λ has color **G**. □

This lemma shows that if there is an occurrence of a node ν with color **G** at the start of a run, but ν is eventually contracted away, the occurrence of color **G** is nevertheless reflected by some persisting node. This ensures that the RA does not "underaccept." But what do the colors signify?

In a run, an occurrence of a node λ with color **Y** indicates an occurrence of λ with color **G**. A leaf λ of color **G** has a macrostate that is a subset of A_Q, and if node λ persists throughout a run, is a leaf at the start of the run, and has color **G** at the end of the run, then the $\mathcal{B}$-runs extracted from the node chain due to λ all enter an accepting state. The next lemmas show how the color **G** relates to $\mathcal{B}$-runs that enter an accepting state.

LEMMA 3.23. *Let* $(\tau, a, \tau') \in \Delta_R$.

(1) If τ' has a node with ID λ and color **G***, and $mstate_\lambda(\tau')$ is* not *a subset of A_Q, then τ has a node ν with color* **G** *or* **Y** *that is contracted into λ on update by a. For any such ν, $mstate_\nu(\tau) \trianglerighteq_a mstate_\lambda(\tau')$.*

(2) If τ' has a node with ID λ and color **Y***, then τ has a node with ID λ and color* **G** *or* **Y**.

(3) In either case, any node occurring in τ' above λ has an ID that appears in τ.

PROOF. For the first claim, let $\tilde{\tau}$ be the tree obtained during $update_a$ prior to the normalization step. Now, the only nodes that have color **G** in $\tilde{\tau}$ are the newly created leaf nodes whose macrostates are subsets of A_Q. If λ is such a node, then $mstate_\lambda(\tau')$ is a subset of A_Q, so suppose that λ is the ID of a node found in τ.

During normalization, a node is deleted at Step 9(c) only when its macrostate is the same as its parent's. Node λ having color **G** in τ' means that in $\tilde{\tau}$, the macrostate of λ is the same as that of a node ν strictly below it; the color of ν is **G** or **Y**, and any node above ν and strictly below λ has an ID that is absent from τ'. If ν is not a node of τ, then it is a newly created node with color **G** and $mstate_\lambda(\tau')$ is a subset of A_Q. Otherwise, by definition, ν is contracted into λ on update of τ by a. For any such ν, $mstate_\nu(\tilde{\tau}) = mstate_\lambda(\tilde{\tau})$. Since $mstate_\nu(\tau) \trianglerighteq_a mstate_\nu(\tilde{\tau})$ and $mstate_\lambda(\tilde{\tau}) = mstate_\lambda(\tau')$, it follows that $mstate_\nu(\tau) \trianglerighteq_a mstate_\lambda(\tau')$.

The other claims are relatively clear. □

The following is an important supporting lemma of Theorem 3.17. It can be used to show that an accepting $\mathcal{R}$-run on w implies some accepting $\mathcal{B}$-run on w. A proof of the converse would then establish Theorem 3.17, an important result in ω-automaton theory.

LEMMA 3.24. *Consider an $\mathcal{R}$-run $\varsigma = \tau_0\tau_1 \ldots \tau_T$ on $w = a_1 \ldots a_T$, where each τ_t has a node with ID λ. Let $r \in mstate_\lambda(\tau_T)$. Then the following hold.*

(1) *There exists a $\mathcal{B}$-run $\varrho = r_0 r_1 \dots r_T$ on w such that each $r_t \in mstate_\lambda(\tau_t)$ and $r_T = r$.*
(2) *If $\tau_0 = \tau_T$ and node λ of τ_T has color* **G***, there exists a $\mathcal{B}$-run $\varrho = r_0 r_1 \dots r_T$ on w such that each $r_t \in mstate_\lambda(\tau_t)$, $r_T = r$, and ϱ enters an accepting state.*

PROOF. The first claim follows easily from Lemmas 3.20(1) and 3.19.

For the second claim, suppose that $\tau_0 = \tau_T$ and the node λ of τ_T has color **G**. Derive an ID sequence $\lambda_0 \lambda_1 \dots \lambda_T$ as follows. First, let $\lambda_T = \lambda$. For $t > 0$, define λ_{t-1} in terms of λ_t: If τ_{t-1} has a node ν with color **G** or **Y** that is contracted into λ_t on update by a_t, then let λ_{t-1} be such a ν (in this case, λ_t necessarily has color **G** in τ_t, by Lemma 3.22), else if λ_t is a node of τ_{t-1}, let $\lambda_{t-1} = \lambda_t$, else let λ_{t-1} be the ID of the parent of λ_t in τ_t. By Lemmas 3.20 and 3.23(1), this is possible and the resulting sequence is a node chain over ς, w.

To show that $mstate_{\lambda_t}(\tau_t) \subseteq A_Q$ for some $t > 0$, suppose the contrary. Since λ has color **G** in τ_T, by Lemma 3.23(1), a node ν of τ_{T-1}, which has color **G** or **Y**, is contracted into λ_T on the update of τ_{T-1} by a_T. By definition, λ_{T-1} is such a ν. Consider any t with $0 < t < T$ such that node λ_t has color **G** or **Y** in τ_t and occurs below λ_{T-1}. Then by Lemma 3.23, the definition of λ_{t-1}, and the assumption that $mstate_{\lambda_t}(\tau_t) \not\subseteq A_Q$, the same holds for node λ_{t-1} in τ_{t-1}. It follows by induction that λ_0 has color **G** or **Y** in τ_0 and occurs below λ_{T-1}. In particular, τ_0 has a node with ID λ_{T-1}. Since λ_{T-1} is contracted into λ_T on the update of τ_{T-1} by a_T, the ID λ_{T-1} cannot appear in τ_T, but $\tau_T = \tau_0$, which is a contradiction.

The next claim is that each λ_t of the node chain occurs in τ_t below λ. This holds for $t = T$, since $\lambda_T = \lambda$. Consider any $t > 0$ for which the claim holds. By definition, $\lambda_{t-1} = \lambda_t$, or λ_{t-1} occurs in τ_{t-1} below λ_t, or τ_{t-1} does not have a node with ID λ_t and λ_{t-1} is the ID of the parent of λ_t in τ_t. In the last case, $\lambda_t \neq \lambda$, so in τ_t, the parent of λ_t occurs (nonstrictly) below λ. In every case, by the definition of *update*, λ_{t-1} occurs in τ_{t-1} below λ. The claim follows by induction.

Finally, apply Lemma 3.19 to deduce the existence of a $\mathcal{B}$-run $\varrho = r_0 \dots r_T$ on w such that each $r_t \in mstate_{\lambda_t}(\tau_t)$ and $r_T = r$. It has been shown that for some $t > 0$, $mstate_{\lambda_t}(\tau_t) \subseteq A_Q$, which implies that $r_t \in A_Q$. Thus, ϱ enters an accepting state. It has also been shown that each λ_t occurs in τ_t below λ, and this implies that $mstate_{\lambda_t}(\tau_t) \subseteq mstate_\lambda(\tau_t)$. Thus, each $r_t \in mstate_\lambda(\tau_t)$. □

Applying the Muller-Schupp procedure to $\mathcal{B}^{de}$. Returning the problem of constructing ranking functions, recall the following definition.

$$DESC = \{cs_0 cs_1 \mid cs_0 \in Sites^*,\ \mathcal{M}_{cs_1} \text{ has complete thread with infinite descent}\}$$

The BA constructed for *DESC* was $\mathcal{B}^{de} = (Q^{de}, Sites, q_0^{de}, \Delta^{de}, A^{de})$, where q_0^{de} is a distinguished start state, $Q^{de} = (Par \times \{\downarrow, \ddagger\}) \cup \{q_0^{de}\}$. Accepting states are given by $A^{de} = Par \times \{\downarrow\}$, and the transition relation Δ^{de} contains:

—(q_0^{de}, c, q_0^{de}) for $c \in Sites$,
—$(q_0^{de}, c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ for some y, and
—$((y, \delta), c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ and $\delta \in \{\downarrow, \ddagger\}$.

Consider the determinization of this BA by the Muller-Schupp procedure. Denote by $\mathcal{R}^{de}$ the resulting RA. The following lemmas constitute a specialization of Lemma 3.24 to $\mathcal{R}^{de}$.

LEMMA 3.25. *Consider a run $\varsigma = \tau_0\tau_1 \dots \tau_T$ of $\mathcal{R}^{de}$, where $\tau_0 = \tau_T$. Assume that each element of ς has a node with ID λ, and the node λ of τ_T has color* **G**. *Let $0 < t \leq T$. Then any $r \in mstate_\lambda(\tau_t)$ has the form (x, γ), where $x \in Par$ and $\gamma \in \{\downarrow, \ddagger\}$, and the sets $P_t = \{x \mid (x, \gamma) \in mstate_\lambda(\tau_t)\}$ are not empty.*

PROOF. By Lemma 3.24(2), for any $q \in mstate_\lambda(\tau_T)$, there is a run of $\mathcal{B}^{de}$ entering an accepting state and ending with q, so q is reachable from an accepting state. Let $0 < t \leq T$. By Lemma 3.24(1), for $t < T$ and $r \in mstate_\lambda(\tau_t)$, there is a run of $\mathcal{B}^{de}$ from some $q \in mstate_\lambda(\tau_0) = mstate_\lambda(\tau_T)$ ending with r, so r is reachable from an accepting state. It follows from the definition of $\mathcal{B}^{de}$ that any $r \in mstate_\lambda(\tau_t)$ has the form (x, γ), where $x \in Par$ and $\gamma \in \{\downarrow, \ddagger\}$. The set P_t is not empty as $mstate_\lambda(\tau_t)$ is not empty. □

LEMMA 3.26. *Let $\varsigma = \tau_0\tau_1 \dots \tau_T$ be a run of $\mathcal{R}^{de}$ on $cs = c_1 \dots c_T$ as described in the previous lemma. Thus, each element of ς has a node with ID λ. Consider a subrun $\tau_{t_0} \dots \tau_{t_1}$, where $t_0 < t_1$. Let $cs' = c_{t_0+1} \dots c_{t_1}$. For each value of t, let $P_t = \{x \mid (x, \gamma) \in mstate_\lambda(\tau_t)\}$. Then the following hold.*

(1) For every $x \in P_{t_1}$, there exists $y \in P_{t_0}$ such that $\mathcal{M}_{cs'}$ has a complete thread from y to x.

(2) If $\tau_{t_0} = \tau_{t_1}$ and the node λ of τ_{t_1} has color **G**, *then for every $x \in P_{t_1}$, there exists $y \in P_{t_0}$ such that $\mathcal{M}_{cs'}$ has a complete and descending thread from y to x.*

PROOF. By Lemma 3.24(1), for any $(x, \gamma) \in mstate_\lambda(\tau_{t_1})$, there is some run $\varrho = r_{t_0} \dots r_{t_1}$ of $\mathcal{B}^{de}$ on cs' such that each $r_t \in mstate_\lambda(\tau_t)$ and $r_{t_1} = (x, \gamma)$. By Lemma 3.25, each r_t has the form (x_t, γ_t), so ϱ corresponds to a complete thread in $\mathcal{M}_{cs'}$ from some y to x, where $y \in P_{t_0}$.

For the second claim, suppose that $\tau_{t_0} = \tau_{t_1}$ and the node λ of τ_{t_1} has color **G**. By Lemma 3.24(2), for any $(x, \gamma) \in mstate_\lambda(\tau_{t_1})$, there is a run $\varrho = r_{t_0} \dots r_{t_1}$ of $\mathcal{B}^{de}$ on cs' such that each $r_t \in mstate_\lambda(\tau_t)$, $r_{t_1} = (x, \gamma)$, and ϱ enters an accepting state. By Lemma 3.25, each r_t has the form (x_t, γ_t), so ϱ corresponds to a complete and descending thread in $\mathcal{M}_{cs'}$ from some y to x, where $y \in P_{t_0}$. □

The next theorem assumes only that $\mathcal{G}$ satisfies SCT. It supports the main result in this article and easily implies, in the case of a single-function $\mathcal{G}$, the existence of an (n, k)-progression on any finite call-site sequence, for bounded values of n and k.

Definition 3.27. Let $\mathcal{R} = (R, \Sigma, q_0, \Delta_R, \Omega_R)$ be a given RA with $R' \subseteq R$ and $\Delta' \subseteq \Delta_R$. Let $\varsigma = r_0r_1r_2 \dots$ be a (finite or infinite) run of $\mathcal{R}$ on $w = a_1a_2 \dots$.

(1) Say that ς *remains in* R', Δ' if $r_t \in R'$ for each t, and $(r_{t-1}, a_t, r_t) \in \Delta'$ for each $t > 0$.

(2) The *restriction of* Ω_R *to* R' is the set $\Omega_{R'} = \{(E, F) \in \Omega_R \mid F \cap R' \neq \{\}\}$.

THEOREM 3.28. *Assume that $\mathcal{G}$ is SCT. Write $(R, Sites, \tau_*, \Delta_R, \Omega_R)$ for the result of applying the Muller-Schupp determinization procedure to $\mathcal{B}^{de}$. Let $R' \subseteq R$ and $\Delta' \subseteq \Delta_R \cap (R' \times Sites \times R')$.*

Suppose that R' is strongly connected by Δ' and every infinite run remaining in R', Δ' is tail-accepting. Consider any $cs = c_1 \dots c_T$ such that there exists a run ς on cs remaining in R', Δ'. Let $m = |R'|$, $n = 2m^2$, and $k = 2|\Omega_{R'}|$. Then there exists an (n, k)-progression on cs.

PROOF. First, argue that Ω_R contains a pair (E, F) with $E \cap R' = \{\}$ and $F \cap R' \neq \{\}$. Suppose the contrary. Let $E' = \bigcup\{E \cap R' \mid (E, F) \in \Omega_R\}$. Since R' is strongly connected by Δ', there is a run $\tau_0 \dots \tau_T$ that remains in R', Δ', starts and ends at the same state, and passes through every element of E'. If the infinite run $(\tau_0 \dots \tau_{T-1})^\omega$ contains any element of F for $(E, F) \in \Omega_R$, then this element is in $F \cap R'$, so $F \cap R' \neq \{\}$. By assumption, $E \cap R' \neq \{\}$. By construction, the infinite run has infinitely many occurrences of each element of $E \cap R'$. Therefore, it has infinitely many occurrences of some element of E. It follows that the infinite run cannot be tail-accepting, which violates the theorem's premise.

Pick an (E, F) from Ω_R with $E \cap R' = \{\}$ and $F \cap R' \neq \{\}$, and let (E, F) be indexed by λ. The emptiness of $E \cap R'$ means that every tree in R' has a node with ID λ. By definition, every tree in F has a node with ID λ and color **G**. Thus, for any run $\tau_0 \tau_1 \dots \tau_T$ that remains in R', Δ', where $\tau_0 = \tau_T \in F$, the conditions of Lemmas 3.25 and 3.26 are satisfied.

This theorem considers a run ς that remains in R', Δ'. The run is labeled by cs. Since R' is strongly connected by Δ', given $\tau \in F \cap R'$, ς can be extended in either or both directions to obtain a run $\tilde{\varsigma}$ that remains in R', Δ', and starts and ends with τ. This run is labeled by some $\widetilde{cs}$ that includes cs as a subword. If there exists an (n, k)-progression on $\widetilde{cs}$, an (n, k)-progression on cs can be extracted. Therefore, without loss of generality, assume that ς itself begins and ends with τ. Specifically, let $\varsigma = \tau_0 \tau_1 \dots \tau_T$, where $\tau_0 = \tau_T = \tau \in F \cap R'$. The theorem is proved by defining an (n, k)-progression on cs, a labeling of this ς.

The induction. Perform induction on the value of k. Assume that the theorem holds for any R'' in place of R' such that $2|\Omega_{R''}| < k$, and show that it holds for R'. In particular, the induction hypothesis applies to $R'' = C_1, \dots, C_N$, the SCCs of $R' \setminus F$ computed with respect to Δ'. Some terminology: Call $(\tau_0, c, \tau_1) \in \Delta'$ *intra-component* if τ_0 and τ_1 are in the same SCC C_i, and *non-intra-component* otherwise.

To use the induction assumption, begin by decomposing ς into consecutive subruns $\varsigma_0, \dots, \varsigma_K$, where each ς_i ends with its first repeat occurrence of an element of F, except for the final subrun ς_K, which need not contain a repeat of any element of F. Since ς begins and ends with $\tau \in F$, there must be, at least, one nonfinal ς_i. Let each ς_i be labeled by cs_i such that $cs = cs_0 \dots cs_K$. A little counting: The number N of SCCs is at most $m - 1$, so any subrun of ς containing $m - 1$ non-intra-component transitions must contain some element of F as the target of one such transition; any subrun containing $(m + 1)(m - 1) = m^2 - 1$ non-intra-component transitions must contain at least $m + 1$ occurrences of elements of F and hence two occurrences of some element. It follows that each ς_i has strictly fewer than m^2 non-intra-component transitions.

To apply Lemma 3.26 to $\varsigma_i = \tau_{t'} \ldots \tau_{t''}$, let $P_t = \{z \mid (z, \gamma) \in mstate_\lambda(\tau_t)\}$ and $x \in P_{t''}$. If ς_i has no repeat occurrence of any element of F, it is the final subrun ς_K. By Lemma 3.26(1), there exists a complete thread from some $y \in P_{t'}$ to x in $\mathcal{M}_{cs_K}$. Otherwise, either $\tau_{t'} = \tau_{t''} \in F$ or there exists t such that $t' < t < t''$ and $\tau_t = \tau_{t''} \in F$. In the first case, Lemma 3.26(2) implies a complete and descending thread from some $y \in P_{t'}$ to x in $\mathcal{M}_{cs_i}$. In the second case, it implies a complete and descending thread from some $z \in P_t$ to x in $\mathcal{M}_{c_{t+1} \ldots c_{t''}}$, while Lemma 3.26(1) implies a complete thread from some $y \in P_{t'}$ to z in $\mathcal{M}_{c_{t'+1} \ldots c_t}$. Either way, $\mathcal{M}_{cs_i}$ has a complete and descending thread from some $y \in P_{t'}$ to x. Working backwards from $\mathcal{M}_{cs_K}$ to $\mathcal{M}_{cs_0}$, a complete thread $\vartheta = x_0 \xrightarrow{\gamma_1} x_1 \ldots \xrightarrow{\gamma_T} x_T$ of $\mathcal{M}_{cs}$ can be defined to end at any given $x_T \in P_T$, and such that the segment of ϑ in each $\mathcal{M}_{cs_i}$ is descending for $i \neq K$.

Define a sequence of integers $\alpha_0, \ldots, \alpha_T$ inductively as follows. Let $\alpha_0 = (n-1)$. For $0 < t \leq T$, if $\gamma_t = \downarrow$, let $\alpha_t = (n-1)$, else if τ_{t-1} and τ_t are in the same SCC of $R' \setminus F$, let $\alpha_t = \alpha_{t-1}$, otherwise let $\alpha_t = \alpha_{t-1} - 1$. Now argue that every $\alpha_t \geq 0$. Suppose the contrary. Then there exist t' and t'' with $t' < t''$ such that $\varsigma' = \tau_{t'} \ldots \tau_{t''}$ involves n non-intra-component transitions and the part of ϑ from t' to t'' contains no strict arcs. Let $\vartheta' = x_{t'} \xrightarrow{\mp} x_{t'+1} \ldots \xrightarrow{\mp} x_{t''}$. By the counting argument presented earlier, the existence of $n = 2m^2$ non-intra-component transitions in ς' means that ς' covers some nonfinal ς_i completely. It follows that the part of ϑ in $\mathcal{M}_{cs_i}$, which is part of ϑ', has no strict arcs, but this contradicts the definition of ϑ.

Having established that $\alpha_0, \ldots, \alpha_T$ are elements of Par_n, define a sequence of *pairs* of Par_n elements as follows. For $0 \leq t \leq T$, let $\vec{z}^{(t,0)} = \langle x_t, \alpha_t \rangle$, where x_t is from the thread ϑ. Then for $0 < t \leq T$, it either holds that $\vec{z}^{(t,0)} \prec_{c_t} \vec{z}^{(t-1,0)}$, or τ_{t-1} and τ_t are in the same SCC of $R' \setminus F$, graph G_{c_t} has the arc $x_{t-1} \xrightarrow{\mp} x_t$, and $\alpha_t = \alpha_{t-1}$.

Now appeal to the induction hypothesis to handle the intra-component transitions of ς. If $R'' = C_i$ is a nontrivial SCC of $R' \setminus F$ and ς' any subrun of ς remaining in R'', Δ', then $|\Omega_{R''}| < |\Omega_{R'}|$, and the induction hypothesis implies an (n, k')-progression on any labeling of ς', where $k' \leq k-2$. Such a progression can be padded with trailing zeros, if necessary, to form an $(n, k-2)$-progression. For each maximal subrun $\tau_{t'} \ldots \tau_{t''}$ of ς containing only elements of C_i, let $\vec{z}^{(t',1)}, \ldots, \vec{z}^{(t'',1)}$ be an $(n, k-2)$-progression on $c_{t'+1} \ldots c_{t''}$. Let $\vec{z}^{(t,1)}$ be a $(k-2)$-tuple of 0 if t is not covered by some subrun and C_i.

Finally, for $0 \leq t \leq T$, define $\vec{z}^{(t)}$ to be the concatenation of $\vec{z}^{(t,0)}$ and $\vec{z}^{(t,1)}$, a k-tuple. Clearly each $\vec{z}^{(t)}$ is an element of Par_n^k. The claim is that $\vec{z}^{(0)}, \ldots, \vec{z}^{(T)}$ is an (n, k)-progression on cs. Let $\vec{z}^{(t-1)} = \langle z_1, \ldots, z_k \rangle$ and $\vec{z}^{(t)} = \langle z'_1, \ldots, z'_k \rangle$ with $0 < t \leq T$. If τ_{t-1} and τ_t of ς belong to different SCCs of $R' \setminus F$, it has been shown that $\langle z'_1, z'_2 \rangle \prec_{c_t} \langle z_1, z_2 \rangle$. It follows that $\vec{z}^{(t)} \prec_{c_t} \vec{z}^{(t-1)}$. If τ_{t-1} and τ_t are in the same SCC C_i, it has been shown that either $\langle z'_1, z'_2 \rangle \prec_{c_t} \langle z_1, z_2 \rangle$, or $z_1 \xrightarrow{\mp} z'_1 \in G_{c_t}$ and $z'_2 = z_2$. Moreover, as part of an $(n, k-2)$-progression for some subrun remaining in C_i, Δ', the relation $\langle z'_3, \ldots, z'_k \rangle \prec_{c_t} \langle z_3, \ldots, z_k \rangle$ holds. It follows that $\vec{z}^{(t)} \prec_{c_t} \vec{z}^{(t-1)}$. This completes the inductive argument and the proof. □

THEOREM 3.29. *Assume that $\mathcal{G}$ is SCT and there is only one program function f. Let $\mathcal{R}^{de} = (R, Sites, \tau_*, \Delta_R, \Omega_R)$, $m = |R|$, $n = 2m^2$, and $k = 2|\Omega_R|$. Suppose*

that cs is a finite call-site sequence. Then there exists an (n, k)*-progression on cs.*

PROOF. Since $\mathcal{G}$ is SCT and there is only one program function, the set *DESC* contains every infinite call-site sequence. Thus, $\mathcal{B}^{de}$ and $\mathcal{R}^{de}$ are universal, i.e., they accept every infinite word. By construction, $\mathcal{R}^{de}$ is deterministic and every $\tau \in R$ is reachable from τ_*. It follows that given a state τ and an infinite call-site sequence, there is a tail-accepting run from τ labeled by the call-site sequence. Another consequence is that every infinite run of the RA is tail-accepting.

Let R' be a *sink component* of $\mathcal{R}^{de}$, an SCC of states such that if $(\tau, c, \tau') \in \Delta_R$ and $\tau \in R'$, then $\tau' \in R'$ [Cormen et al. 2001]. Fix τ to be an element of R'. Since there is a run that starts with τ, e.g., the one labeled by cs^ω, the component R' is non-trivial and strongly connected. Let $\Delta' = \Delta_R \cap (R' \times \mathit{Sites} \times R')$. Then R' is strongly connected by Δ', every infinite run remaining in R', Δ' is tail-accepting, and for the call-site sequence cs, there is a run from τ that is labeled by cs and remains in R', Δ'. By Theorem 3.28 and the fact that $|\Omega_{R'}| \leq |\Omega_R|$, there exists an (n', k')-progression on cs, where $n' \leq n$ and $k' \leq k$. By padding each element of the progression with trailing zeros, if necessary, an (n, k)-progression on cs is obtained. □

COROLLARY 3.30. *Assume that* $\mathcal{G}$ *is SCT and there is only one program function* f*. Let* $\mathcal{R}^{de} = (R, \mathit{Sites}, \tau_*, \Delta_R, \Omega_R)$*,* $m = |R|$*,* $n = 2m^2$*, and* $k = 2|\Omega_R|$*. Then* $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid Z \in \mathcal{S}_{n,k}\}$ *specifies a* $\mathcal{G}$*-ranking function, where* $\mathcal{S}_{n,k}$ *is the closure defined in Lemma 3.13.*

PROOF. By the previous theorem, there exists an (n, k)-progression on any finite call-site sequence. By Lemma 3.15, the closure $\mathcal{S}_{n,k}$ does not contain the empty set. By Lemma 3.13, the ρ_f stated is a $\mathcal{G}$-ranking function. □

As the progression defined in the proof of Theorem 3.28 has tuples of a particular form, where odd positions contain parameters or zeros and even positions contain numbers, only such tuples need to be considered for the construction of a $\mathcal{G}$-ranking function. For n and k as given in Theorem 3.29, the resulting function can be regarded as mapping into a well-ordered set $(D \times \alpha)^{(k/2)}$, where α is a chain of n elements. To appreciate how large the expression for ρ_f in Corollary 3.30 might become, note that it is a subset of $\wp(\mathit{Par}_n^k)$, whose size is doubly exponential in k.

4. RANKING FUNCTIONS FOR SIZE-CHANGE TERMINATION

It remains to account for the subject program's flowchart, which constrains the call-site sequences of interest to those that are control-flow legal.

4.1 Strongly Connected Flowchart

Let $\mathcal{G}$ be a given set of size-change graphs. The flowchart that follows is implied by $\mathcal{G}$.

Definition 4.1. The *flowchart of* $\mathcal{G}$ is the digraph whose node set is *Fun* and whose arc set contains (g, f) if $g \xrightarrow{G_c} f$ is in $\mathcal{G}$.

Suppose that $\mathcal{G}$ is SCT and assume, for now, that its flowchart is strongly connected. Under these assumptions, a $\mathcal{G}$-ranking function can be constructed with very minor adjustments to the approach used for a single-function $\mathcal{G}$ in Section 3.2. To understand the abbreviated exposition below, the development in that subsection should be consulted.

Definition 4.2. (Modification to $\mathcal{S}_{n,k}$) The notation $\vec{z}' \prec_c \vec{z}$ for $\vec{z}, \vec{z}' \in Par_n^k$ and $c \in Sites$ was introduced in Definition 3.8 and used to formulate (n,k)-progressions in Definition 3.14. In order to reuse Theorem 3.28 without modifying its (somewhat intricate) proof, the revised $\mathcal{S}_{n,k}$ closure will be based on the same $\prec_c$ relation (see Definition 3.8). However, the new closure set has to account for the flowchart of $\mathcal{G}$.

As before, for $c \in Sites$ and $Z \subseteq Par_n^k$, let $down_c(Z) = \{\vec{z}' \mid \vec{z}' \prec_c \vec{z},\ \vec{z} \in Z\}$, but $\mathcal{S}_{n,k}$ is now the least set that satisfies the following:

—$(Par_n^k, f) \in \mathcal{S}_{n,k}$ for $f \in Fun$, and
—if $(Z, f) \in \mathcal{S}_{n,k}$ and $c : f \to f'$, then $(down_c(Z), f') \in \mathrm{S}_{n,k}$.

When evaluating a tuple $\vec{z} \in Par_n^k$ with respect to program state $(f, \vec{v})$, any parameter occurring in $\vec{z}$ but not belonging to $Param(f)$ will be treated as 0.

Definition 4.3. (Valuation of tuples) Let $s = (f, \vec{v}) \in St$ be any program state. For $\vec{z} = \langle z_1, \ldots, z_k \rangle \in Par_n^k$, the valuation of $\vec{z}$ in s is the k-tuple $\langle v_1, \ldots, v_k \rangle$, where for $1 \le i \le k$, if $z_i \in Param(f)$, then v_i is the value of z_i in s; if z_i is a parameter not belonging to $Param(f)$, then $v_i = (0)_D$, the least element of D; and if z_i is a number, then $v_i = (z_i)_D$, the z_i-th smallest element of D.

The result of the valuation is an element of D^k. Recall that D^k is well-ordered by the lexicographic extension of $\le_D$, and the maximum over a finite subset of D^k is taken with respect to this well-ordering. As in Definition 3.9, for $Z \subseteq Par_n^k$, define $\textsc{maxval}_Z : St \to D^k$ as follows.

$$\textsc{maxval}_Z(s) = \max\{\vec{v} \mid \vec{z} \in Z,\ \vec{v} \text{ is the valuation of } \vec{z} \text{ in } s\}.$$

The reader may like to verify that the proofs of Lemma 3.10 and 3.11 apply to the following claims without any modification.

Lemma 4.4. *Let $\vec{z} = \langle z_1, \ldots, z_k \rangle$ and $\vec{z}' = \langle z'_1, \ldots, z'_k \rangle$ be elements of Par_n^k with $\vec{z}' \prec_c \vec{z}$. Suppose that $s \xrightarrow{c}_{\mathcal{G}} s'$. Let $\vec{v} = \langle v_1, \ldots, v_k \rangle$ be the valuation of $\vec{z}$ in s and $\vec{v}' = \langle v'_1, \ldots, v'_k \rangle$ the valuation of $\vec{z}'$ in s'. Then $\vec{v}' < \vec{v}$.*

Lemma 4.5. *Suppose that $Z, Z' \subseteq Par_n^k$, $Z' \subseteq down_c(Z)$, Z' is nonempty, and $s \xrightarrow{c}_{\mathcal{G}} s'$. Then $\textsc{maxval}_{Z'}(s') < \textsc{maxval}_Z(s)$.*

Below is the updated version of Lemma 3.12.

Lemma 4.6. *Suppose that $\mathcal{S} \subseteq \wp(Par_n^k) \times Fun$ contains an element of the form (Z, f) for each $f \in Fun$, and for any $(Z, f) \in \mathcal{S}$ and call-site $c : f \to f'$, there exists $(Z', f') \in \mathcal{S}$ such that Z' is nonempty and a subset of $down_c(Z)$. Then $\rho_f(s) = \min\{\textsc{maxval}_Z(s) \mid (Z, f) \in \mathcal{S}\}$ specifies a $\mathcal{G}$-ranking function.*

PROOF. Let $c : f \to f'$ and $s \xrightarrow{c}_{\mathcal{G}} s'$. By assumption, given $(Z, f) \in \mathcal{S}$, there exists $(Z', f') \in \mathcal{S}$ such that Z' is nonempty and a subset of $down_c(Z)$. By Lemma 4.5, $\text{MAXVAL}_{Z'}(s') < \text{MAXVAL}_Z(s)$. Clearly, the value of $\rho_{f'}(s')$ is no greater than $\text{MAXVAL}_{Z'}(s')$. It follows that $\rho_{f'}(s') < \text{MAXVAL}_Z(s)$. Since $\mathcal{S}$ has an element of the form (Z, f), there is a *particular* (Z, f) for which $\rho_f(s) = \text{MAXVAL}_Z(s)$. Based on this instance of (Z, f), $\rho_{f'}(s') < \rho_f(s)$. Therefore, ρ is a $\mathcal{G}$-ranking function. □

LEMMA 4.7. *Assume that there exist n and k such that for every* $(Z, f) \in \mathcal{S}_{n,k}$, *the set Z is nonempty. Then* $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid (Z, f) \in \mathcal{S}_{n,k}\}$ *specifies a* $\mathcal{G}$*-ranking function.*

PROOF. By definition, $\mathcal{S}_{n,k}$ has an element of the form (Z, f) for each $f \in Fun$. By the assumption in the lemma and the definition of $\mathcal{S}_{n,k}$, for every $(Z, f) \in \mathcal{S}_{n,k}$ and call-site $c : f \to f'$, there exists $(Z', f') \in \mathcal{S}_{n,k}$ such that $Z' = down_c(Z)$ and Z' is nonempty. By Lemma 4.6, ρ is a $\mathcal{G}$-ranking function. □

It remains to discharge the assumption in the previous lemma.

LEMMA 4.8. *Assume that there is an* (n, k)*-progression on any finite, control-flow legal call-site sequence. Then for every* $(Z, f) \in \mathcal{S}_{n,k}$, *Z is nonempty.*

PROOF. For any $(Z, f) \in \mathcal{S}_{n,k}$, if $Z \neq Par_n^k$, there exist a control-flow legal call-site sequence $cs = c_1 \ldots c_T$ and a sequence $(Z_0, f_0), (Z_1, f_1), \ldots, (Z_T, f_T)$ of elements of $\mathcal{S}_{n,k}$ such that $Z_0 = Par_n^k$, $(Z_T, f_T) = (Z, f)$, and for $0 < t \leq T$, it holds that $c_t : f_{t-1} \to f_t$ and $Z_t = down_{c_t}(Z_{t-1})$. By assumption, there is an (n, k)-progression $\vec{z}^{(0)}, \vec{z}^{(1)}, \ldots, \vec{z}^{(T)}$ on cs, where $\vec{z}^{(t)} \prec_{c_t} \vec{z}^{(t-1)}$ for $0 < t \leq T$. By induction, $\vec{z}^{(T)} \in Z_T$. Therefore, Z is nonempty. □

The following is analogous to Theorem 3.29.

THEOREM 4.9. *Assume that* $\mathcal{G}$ *is SCT and the flowchart of* $\mathcal{G}$ *is strongly connected. Let* $\mathcal{R}^{de} = (R, Sites, \tau_*, \Delta_R, \Omega_R)$ *be the RA obtained by applying the Muller-Schupp procedure to* $\mathcal{B}^{de}$, *the BA that accepts DESC. Let* $m = |R|$, $n = 2m^2$, *and* $k = 2|\Omega_R|$. *Suppose that* $\widetilde{cs}$ *is a finite call-site sequence that is control-flow legal. Then there is an* (n, k)*-progression on* $\widetilde{cs}$.

PROOF. Since $\mathcal{G}$ is SCT, *DESC* accepts every infinite call-site sequence of the form $cs_0\, cs_1$, where cs_0 is arbitrary, but cs_1 is control-flow legal. By construction, $\mathcal{R}^{de}$ is deterministic and every $\tau \in R$ is reachable from τ_*. It follows that for any $\tau \in R$ and $cs \in Sites^\omega$ that is control-flow legal, there is a tail-accepting run from τ labeled by cs. Another consequence is that every infinite run of $\mathcal{R}^{de}$ labeled by a control-flow legal call-site sequence is tail-accepting.

Next, recall the transitions of $\mathcal{B}^{de}$:

(1) (q_0^{de}, c, q_0^{de}) for $c \in Sites$,

(2) $(q_0^{de}, c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ for some y, and

(3) $((y, \delta), c, (x, \gamma))$, if $y \xrightarrow{\gamma} x \in G_c$ and $\delta \in \{\downarrow, \updownarrow\}$.

For $\tau \in R$, denote by Q_τ the macrostate at the root of τ. For any $(\tau, c, \tau') \in \Delta_R$, $Q_{\tau'}$ is the result of applying the subset construction to Q_τ. Since $Q_{\tau_*} = \{q_0^{de}\}$, the transitions specified by (1) means that q_0^{de} is in Q_τ for every $\tau \in R$. As the flowchart of $\mathcal{G}$ is strongly connected, each size-change graph G_c must have at least one arc for $\mathcal{G}$ to be SCT. It follows from the transitions specified by (1)–(3) that if $(\tau, c, \tau') \in \Delta_R$ and $c : f \to f'$, then $Q_{\tau'} \setminus \{q_0^{de}\}$ is a nonempty subset of $Param(f')$. Define $fun_{\tau'}$ to be f', the program function associated with τ'.

Next, let $\Delta = \{(\tau, c, \tau') \in \Delta_R \mid c : f \to f', fun_\tau = f, fun_{\tau'} = f'\}$. Clearly, runs remaining in R, Δ can only be labeled by control-flow legal call-site sequences. In the other direction, if $fun_\tau = f$, and the call-site sequence cs is control-flow legal and begins with some $c : f \to f'$, then any run from τ labeled by cs uses only Δ transitions.

Let R' be a sink component of the restriction of $\mathcal{R}^{de}$ to Δ transitions, and put $\Delta' = \Delta \cap (R' \times Sites \times R')$. Let $\tau \in R'$ and $f = fun_\tau$. If the control-flow legal call-site sequence $\widetilde{cs}$ does not begin with some $c : f \to f'$, it can be extended on the left to one that does, and a progression on $\widetilde{cs}$ can be extracted from a progression on the extended call-site sequence. Therefore, without loss of generality, assume that $\widetilde{cs}$ begins with some $c : f \to f'$. Note that $\widetilde{cs}$ can be extended on the right to an infinite control-flow legal call-site sequence, and there is a run from τ labeled by this call-site sequence using only Δ transitions, so R' is a nontrivial component and strongly connected by Δ'. As infinite runs remaining in R', Δ' can only be labeled by control-flow legal call-site sequences, they are tail-accepting, by an observation made in the opening paragraph of this proof.

Turning back to $\widetilde{cs}$, from what has been noted, there is a run from τ labeled by $\widetilde{cs}$ that uses only Δ transitions and therefore remains in R', Δ'. By Theorem 3.28 and the fact that $|\Omega_{R'}| \leq |\Omega_R|$, there exists an (n', k')-progression on $\widetilde{cs}$, where $n' \leq n$ and $k' \leq k$. Pad the elements of the progression with trailing zeros, as necessary, to obtain an (n, k)-progression. □

COROLLARY 4.10. *Assume that $\mathcal{G}$ is SCT and the flowchart of $\mathcal{G}$ is strongly connected. Let $\mathcal{R}^{de} = (R, Sites, \tau_*, \Delta_R, \Omega_R)$, $m = |R|$, $n = 2m^2$, and $k = 2|\Omega_R|$. Then $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid (Z, f) \in \mathcal{S}_{n,k}\}$ specifies a $\mathcal{G}$-ranking function, where $\mathcal{S}_{n,k}$ is the closure defined in Definition 4.2.*

PROOF. By the previous theorem, there exists an (n, k)-progression on any finite call-site sequence that is control-flow legal. By Lemma 4.8, the closure $\mathcal{S}_{n,k}$ does not contain any (Z, f) where Z is empty. By Lemma 4.7, the given ρ_f specifies a $\mathcal{G}$-ranking function. □

A careful review of all the arguments involved shows that $\mathcal{S}_{n,k}$ can be defined more naturally as follows. Let $P_f = (Param(f) \cup \{0, \ldots, n-1\})^k$, for each program function f. Then let $\mathcal{S}_{n,k}$ be the least set that satisfies the following:

—$(P_f, f) \in \mathcal{S}_{n,k}$ for $f \in Fun$, and
—for any $(Z, f) \in \mathcal{S}_{n,k}$ and call-site $c : f \to f'$, it holds that $(Z', f') \in \mathcal{S}_{n,k}$, where $Z' = \{\vec{z}' \in P_{f'} \mid \vec{z}' \prec_c \vec{z}, \vec{z} \in Z\}$.

In fact, as noted before, the proof of Theorem 3.28 uses tuples of a particular form, where odd positions contain parameters or zeros and even positions contain numbers. Thus, only such tuples need to be considered. However, even with these improvements, the construction of a $\mathcal{G}$-ranking function is clearly still impractical.

4.2 The Complete Solution

Finally, it is possible to present the general construction of a $\mathcal{G}$-ranking function.

THEOREM 4.11. *Let $\mathcal{G}$ satisfy SCT. There is an effective procedure to construct a $\mathcal{G}$-ranking function expressed using only* min*,* max*, and lexicographic tuples of program parameters and constants.*

PROOF. Let $C_1, \ldots, C_N$ be the SCCs of the flowchart of $\mathcal{G}$, sorted in reverse topological order. For any program function f in C_i, let $\pi_f = i$, i.e., π_f denotes the *position* of f's component in the list $C_1, \ldots, C_N$.

The closure $\mathcal{S}_{n,k}$ was defined as the least set such that (Par_n^k, f) is in the set, for $f \in Fun$, and if (Z, f) is in the set and $c : f \to f'$, then $(down_c(Z), f')$ is in the set. Now define $\mathcal{S}_{n,k}^{(i)}$ to be the closure computed with respect to C_i. For each C_i, let $P_i = ((\bigcup_{f \in C_i} Param(f)) \cup \{0, \ldots, n-1\})^k$. Then $\mathcal{S}_{n,k}^{(i)}$ is the least set such that:

—$(P_i, f) \in \mathcal{S}_{n,k}^{(i)}$ for $f \in C_i$, and

—if $(Z, f) \in \mathcal{S}_{n,k}^{(i)}$, and $c : f \to f'$ where $f, f' \in C_i$, then $(down_c(Z), f') \in \mathcal{S}_{n,k}^{(i)}$.

Suppose that C_i is a nontrivial component. By applying Corollary 4.10 to the subset of size-change graphs $\mathcal{G}^{(i)} = \{f \xrightarrow{G_c} f' \text{ in } \mathcal{G} \mid f, f' \in C_i\}$ (and its associated program), it can be concluded that $\rho_f(s) = \min\{\text{MAXVAL}_Z(s) \mid (Z, f) \in \mathcal{S}_{n,k}^{(i)}\}$ specifies a $\mathcal{G}^{(i)}$-ranking function for suitably large n and k. Fix n and k to values large enough for every $\mathcal{G}^{(i)}$ where C_i is a nontrivial component. Note that if C_i is trivial, $\mathcal{S}_{n,k}^{(i)}$ contains a single element of the form (Z, f), where Z is nonempty, and f is the sole member of C_i.

For $\vec{z} = \langle z_1, \ldots, z_k \rangle \in Par_n^k$, let $aug_f(\vec{z})$ denote the $k+1$ tuple $\langle \pi_f, z_1, \ldots, z_k \rangle$. For $Z \subseteq Par_n^k$, let $aug_f(Z)$ denote $\{aug_f(\vec{z}) \mid \vec{z} \in Z\}$. Finally, let ρ be given by:

$$\rho_f(s) = \min\{\text{MAXVAL}_{\tilde{Z}}(s) \mid (Z, f) \in \mathcal{S}_{n,k}^{(\pi_f)},\ \tilde{Z} = aug_f(Z)\}.$$

The claim is that ρ is a $\mathcal{G}$-ranking function. Suppose that $s \xrightarrow{c}_{\mathcal{G}} s'$ and $c : f \to f'$. If f and f' are in different SCCs of the flowchart, then $\pi_{f'} < \pi_f$. By definition of ρ, the evaluation of $\rho_f(s)$ is a tuple whose top position is $(\pi_f)_D$, the π_f-th smallest element of D, while the evaluation of $\rho_{f'}(s')$ is a tuple whose top position is $(\pi_{f'})_D$, the $\pi_{f'}$-th smallest element of D. Hence, $\rho_{f'}(s') < \rho_f(s)$. Otherwise, f and f' are in the same component C_i. Let $\rho_f(s) = \langle v_0, v_1, \ldots, v_k \rangle$ and $\rho_{f'}(s') = \langle v_0', v_1', \ldots, v_k' \rangle$. By construction, it holds that $v_0 = (i)_D = v_0'$, and $\langle v_1', \ldots, v_k' \rangle < \langle v_1, \ldots, v_k \rangle$. Hence, $\rho_{f'}(s') < \rho_f(s)$. □

The result of Ben-Amram [2002] follows easily.

COROLLARY 4.12. *Suppose that $\mathcal{G}$ is safe for subject program p, and $\mathcal{G}$ is SCT. If the primitive operations of the subject language compute multiply recursive functions, then p computes a multiply recursive function.*

5. DISCUSSION AND CONCLUSION

Even though the proposed construction of $\mathcal{G}$-ranking functions is impractical, an understanding of the theory involved and some clever guesswork make it possible to work out interesting ranking functions for simple examples. A ranking function of the suitable form can be verified using Lemma 3.12 or Lemma 4.6.

5.1 Examples Revisited

Example 5.1. Consider the size-change graphs $\mathcal{G}$ seen for Example 1.7. For any state transition $((\mathtt{m}, (x, y)), c, (\mathtt{m}, (x', y')))$ safely described by a graph of $\mathcal{G}$, either $c = 1$ and $y' <_D y$ and $x' \leq_D y$, or $c = 2$ and $y' <_D x$ and $x' \leq_D y$.

It turns out that $\rho_{\mathtt{m}}(x, y) = \max\{\langle y, 1\rangle, \langle x, 0\rangle\}$ is a $\mathcal{G}$-ranking function. This follows from an application of Lemma 3.12 to $\mathcal{S} = \{\{\langle \mathtt{y}, 1\rangle, \langle \mathtt{x}, 0\rangle\}\}$, observing that:

$$\langle \mathtt{y}, 1\rangle \prec_1 \langle \mathtt{y}, 1\rangle, \quad \langle \mathtt{x}, 0\rangle \prec_1 \langle \mathtt{y}, 1\rangle,$$
$$\langle \mathtt{y}, 1\rangle \prec_2 \langle \mathtt{x}, 0\rangle, \quad \langle \mathtt{x}, 0\rangle \prec_2 \langle \mathtt{y}, 1\rangle.$$

Note that the ranking function reflects the length of the longest possible chain of function calls, which is twice the initial value of x or y.

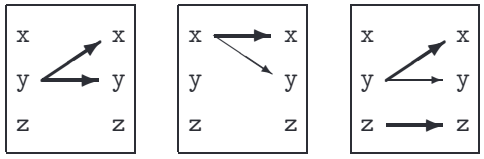
Example 5.2. Consider the graphs $\mathcal{G}$ seen for Example 1.4 (permuted arguments) . For any state transition $((\mathtt{p}, (x, y, z)), c, (\mathtt{p}, (x', y', z')))$ safely described by a graph of $\mathcal{G}$, either $x' \leq_D x$ and $y' <_D z$ and $z' \leq_D y$, or $x' \leq_D z$ and $y' <_D y$ and $z' \leq_D x$.

The example exhibits multiset descent. Since D is well-ordered, this is equivalent to lexicographic descent when comparing tuples of parameters sorted in decreasing order. To express the ranking function in a form that can be verified by Lemma 3.12, let $\rho_{\mathtt{p}}(x, y, z)$ be the maximum over every permutation of x, y, z.

The final example is related to one in Codish et al. [2005]. It was attributed to Ben-Amram.

Example 5.3. (Final mystery descent)

```
m(x,y,z) = if ...
    1m(y-1,y-1,...) else if ...
    2m(x-1,x,...) else
    3m(y-1,y,z-1)
```



G_1 G_2 G_3

For any state transition $((\mathtt{m}, (x, y, z)), c, (\mathtt{m}, (x', y', z')))$ safely described by a graph of the given $\mathcal{G}$, one of the following holds:

—$x' <_D y$ and $y' <_D y$, or
—$x' <_D x$ and $y' \leq_D x$, or
—$x' <_D y$ and $y' \leq_D y$ and $z' <_D z$.

It turns out that $\rho_{\mathtt{m}}(x, y, z) = \max\{\langle y, 0, z\rangle, \langle x, 1, z\rangle\}$ is a $\mathcal{G}$-ranking function. This follows from an application of Lemma 3.12 to $\mathcal{S} = \{\{\langle \mathrm{y}, 0, \mathrm{z}\rangle, \langle \mathrm{x}, 1, \mathrm{z}\rangle\}\}$, observing that:

$$\begin{array}{ll} \langle \mathrm{y}, 0, \mathrm{z}\rangle \prec_1 \langle \mathrm{y}, 0, \mathrm{z}\rangle, & \langle \mathrm{x}, 1, \mathrm{z}\rangle \prec_1 \langle \mathrm{y}, 0, \mathrm{z}\rangle, \\ \langle \mathrm{y}, 0, \mathrm{z}\rangle \prec_2 \langle \mathrm{x}, 1, \mathrm{z}\rangle, & \langle \mathrm{x}, 1, \mathrm{z}\rangle \prec_2 \langle \mathrm{x}, 1, \mathrm{z}\rangle, \\ \langle \mathrm{y}, 0, \mathrm{z}\rangle \prec_3 \langle \mathrm{y}, 0, \mathrm{z}\rangle, & \langle \mathrm{x}, 1, \mathrm{z}\rangle \prec_3 \langle \mathrm{y}, 0, \mathrm{z}\rangle. \end{array}$$

The related example in [Codish et al. 2005] actually leads to the *transposition* of these graphs: The transposition of $G_c : g \to f$ is $G_c^t : f \to g$, where G_c^t has an arc $x \xrightarrow{\gamma} y$ just when G_c has an arc $y \xrightarrow{\gamma} x$, and the transposition of $\mathcal{G}$ is the result of transposing every graph in $\mathcal{G}$.

A $\mathcal{G}$-ranking function for that example is obtained by replacing max with min and swapping 0 and 1 in the expression for $\rho_{\mathtt{m}}$ above. The reader is invited to show that the relationship between these ranking functions is not coincidental!

5.2 Final Remarks

Size-change termination analysis is based on the following strategy. Given the subject program p, first derive a set of size-change graphs $\mathcal{G}$ that represents an overapproximation of the state transition system of p. It turns out to be decidable whether the overapproximation admits an infinite chain of state transitions, by checking a condition on $\mathcal{G}$ called SCT. If $\mathcal{G}$ satisfies SCT, the approximate system does *not* admit any infinite chains, so p is terminating. Otherwise, the information contained in $\mathcal{G}$ does not establish termination of p.

Naturally, there is a ranking function corresponding to any instance of size-change termination. When $\mathcal{G}$ satisfies SCT, this function captures the parameter-descent behavior common to the approximate and original transition systems. Unfortunately, the decision procedure for SCT does not suggest a $\mathcal{G}$-ranking function when it succeeds. This article shows how one can be constructed in principle.

While a suitable ranking function, coupled with bounds for parameter values, can provide valuable information for runtime analysis [Frederiksen 2002], more research is needed for a practical procedure able to construct succinct ranking functions in typical situations.

The purpose of this article has been to investigate the scope of size-change termination. Not only is it the case that any program proved terminating by size-change analysis computes a multiply recursive function, as proved in Ben-Amram [2002], it is, in theory, possible to provide a ranking function that can be used to verify the termination of the subject program by checking lexicographic descent. The ranking function has a particularly simple form. It is expressed using only min, max, and lexicographic tuples of program parameters and constants. This result should be of interest to researchers who make use of the SCT

condition [Frederiksen 2002; Jones and Bohr 2004; Thiemann and Giesl 2003; Wahlstedt 2000] or closely related conditions [Anderson and Khoo 2003; Avery 2005; Codish et al. 2005; Glenstrup and Jones 2004].

A simple description for a space of ranking functions associated with SCT is also useful when comparing size-change analysis with other termination analyses. In an article by Ben-Amram and Lee [2007], ranking functions for a subset of SCT instances are expressed using a combination of lexicographic tuples, multisets, and minimums and maximums over parameters. The use of multisets fails to compensate for not taking minimum and maximum over lexicographic tuples. In fact, Example 5.2 shows that in the context of SCT, multisets help with succinctness, not expressiveness. To sum up the result in this article: Minimums and maximums over lexicographic tuples are all that is necessary to describe the parameter-descent behavior in size-change termination.

Here are some obvious follow-up questions. What is a good set of operations to make succinct ranking functions possible for typical SCT instances? Will simpler ranking functions suffice for size-change graphs of a restricted form? A natural class to consider: graphs that are fan-in free, where every target node has in-degree at most 1. Finally, what is a *practical* procedure for constructing ranking functions for size-change termination?

APPENDIX

A. SEMANTIC PROOFS

The following is a restatement of Lemma 2.4.

LEMMA A.1. *Consider any state $s = (g, \vec{u}) \in St$ and a subexpression e in the body of g. Let $\mathcal{E}[\![e]\!]\vec{u} = w$ and $\mathcal{T}[\![e]\!]\vec{u} = C$. Then $w = \bot$ just when C contains some (s, c, s') with s' divergent.*

PROOF. The bi-implication is proved by structural induction on e.

—Suppose that e is a parameter. Then $w \neq \bot$ and $C = \{\}$, so both sides of the bi-implication are false.

—Suppose that $e = \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3$. For $i = 1, 2, 3$, let $\mathcal{E}[\![e_i]\!]\vec{u} = w_i$ and $\mathcal{T}[\![e_i]\!]\vec{u} = C_i$. There are several cases to consider.
If $w_1 = \bot$, then $w = \bot$ and $C = C_1$. By the induction hypothesis, C_1 contains some (s, c, s') with s' divergent, so both sides of the bi-implication are true.
If $w_1 = \mathit{lift}\ 1$, then $w = w_2$ and $C = C_1 \cup C_2$. By the induction hypothesis, C_1 contains no (s, c, s') with s' divergent. Thus, there exists $(s, c, s') \in C$ with s' divergent just when there exists $(s, c, s') \in C_2$ with s' divergent. By the induction hypothesis, this happens just when $w_2 = \bot$, that is, just when $w = \bot$, so the bi-implication holds. The case for $w_1 = \mathit{lift}\ 0$ is similar.
Finally, for w_1 having any other value, $w = \mathit{Err}$ and $C = C_1$. By the induction hypothesis, C_1 contains no (s, c, s') with s' divergent, so both sides of the bi-implication are false.

—Suppose that $e = op(e_1, \ldots, e_K)$. For each i, let $\mathcal{E}[\![e_i]\!]\vec{u} = w_i$ and $\mathcal{T}[\![e_i]\!]\vec{u} = C_i$. Consider the case where $w_i = \mathit{lift}\ v_i$ for each i, and let $\vec{v} = (v_1, \ldots, v_K)$. Then $w \neq \bot$ (by the assumption that $\mathcal{O}[\![op]\!]\vec{v} \neq \bot$) and $C = \bigcup_i C_i$. By the

induction hypothesis, none of the C_i's contains any (s, c, s') with s' divergent, so both sides of the bi-implication are false.
In the other case, let j be the least index such that $w_j \in \{\bot, \mathit{Err}\}$. Then $w = w_j$ and $C = \bigcup_{i \le j} C_i$. If $w_j = \bot$, it follows from the induction hypothesis that C_j contains some (s, c, s') with s' divergent, so both sides of the bi-implication are true. If $w_j = \mathit{Err}$, it follows from the induction hypothesis that none of the sets $C_1, \ldots, C_j$ contains any (s, c, s') with s' divergent, so both sides of the bi-implication are false.

—Suppose that $e = {}^c f(e_1, \ldots, e_K)$. For each i, let $\mathcal{E}[\![e_i]\!]\vec{u} = w_i$ and $\mathcal{T}[\![e_i]\!]\vec{u} = C_i$. Consider the case where $w_i = \mathit{lift}\, v_i$ for each i, and let $\vec{v} = (v_1, \ldots, v_K)$. Then $w = \mathcal{E}[\![e_f]\!]\vec{v}$ and $C = C_1 \cup \ldots \cup C_K \cup \{((g, \vec{u}), c, (f, \vec{v}))\}$. By the induction hypothesis, none of the sets $C_1, \ldots, C_K$ contains any (s, c, s') with s' divergent. Thus, there exists $(s, c, s') \in C$ with s' divergent just when $(f, \vec{v})$ is divergent, i.e., just when $\mathcal{E}[\![e_f]\!]\vec{v} = \bot$. Since $w = \mathcal{E}[\![e_f]\!]\vec{v}$, this happens just when $w = \bot$, so the bi-implication holds.
In the other case, let j be the least index such that $w_j \in \{\bot, \mathit{Err}\}$, and argue as in the corresponding situation for $e = op(e_1, \ldots, e_K)$. □

It is now straightforward to show that if a program state is divergent, there exists an infinite $\rightarrow_p$-chain that starts with it, as claimed in Lemma 2.5. For any divergent state $s = (f, \vec{v})$, it follows from the previous lemma that $\mathcal{T}[\![e_f]\!]\vec{v}$ contains some (s, c, s') with s' divergent, that is, for any divergent state s, there exist c and s' such that $s \overset{c}{\rightarrow}_p s'$ and s' is divergent. Hence, given a divergent state, it is possible to define, by induction, an infinite $\rightarrow_p$-chain starting with it.

For the converse of Lemma 2.5, recall that the evaluation function $\mathcal{E}$ is the least fixed point of the functional $\mathcal{F}$ defined in Section 2.1, so $\mathcal{E} = \bigsqcup_{n<\omega} \mathcal{F}^n(\mathcal{E}_0)$, where $\mathcal{E}_0[\![e]\!]\vec{v} = \bot$ for all e and $\vec{v}$. Writing $\mathcal{E}_n$ for $\mathcal{F}^n(\mathcal{E}_0)$, the sequence $\mathcal{E}_0, \mathcal{E}_1, \mathcal{E}_2, \ldots$ monotonically increases towards $\mathcal{E}$. For any program state $s = (f, \vec{v})$ such that $\mathcal{E}[\![e_f]\!]\vec{v} \neq \bot$, there exists some n such that $\mathcal{E}_n[\![e_f]\!]\vec{v} \neq \bot$. For any such n, say that s is defined at $\mathcal{E}_n$.

Suppose that e is a subexpression of e_f. It can be proved by structural induction on e that if $\mathcal{E}_n[\![e]\!]\vec{v} \neq \bot$, then for any $(s, c, s') \in \mathcal{T}[\![e]\!]\vec{v}$, the program state s' is defined at $\mathcal{E}_{n-1}$. It follows that if a program state s is defined at $\mathcal{E}_n$, and $s \overset{c}{\rightarrow}_p s'$, then s' is defined at $\mathcal{E}_{n-1}$.

Suppose that $s_0 \overset{c_1}{\rightarrow} s_1 \overset{c_2}{\rightarrow} s_2 \ldots$ is an infinite $\rightarrow_p$-chain but s_0 is *not* divergent. Let s_0 be defined at $\mathcal{E}_n$. Applying the above result repeatedly, for any $i \ge 0$, s_i is defined at $\mathcal{E}_{n-i}$, which implies that $n - i > 0$. For large enough i, this is impossible. Therefore, if there exists an infinite $\rightarrow_p$-chain starting from a program state, then this program state must be divergent.

REFERENCES

Althoff, C. S., Thomas, W., and Wallmeier, N. 2005. Observations on Determinization of Büchi Automata. In *Proceedings of the 10th International Conference on Implementation and Application of Automata, (CIAA'05)*. Springer.

Anderson, H. and Khoo, S. C. 2003. Affine-based size-change termination. In *Proceedings of the 1st Asian Symposium on Programming Languages and Systems (APLAS'03)*, Ohori, Ed. Lecture Notes in Computer Science, vol. 2895. Springer, 122–140.

AVERY, J. 2005. The size-change termination principle on non well founded data types. Tech. Rep., DIKU, Denmark.

BEN-AMRAM, A. 2002. General size-change termination and lexicographic descent. In *The Essence of Computation: Complexity, Analysis, Transformation. Essays Dedicated to Neil D. Jones*, T. Mogensen, D. Schmidt, and H. Sudborough, Eds. Lecture Notes in Computer Science, vol. 2566. Springer, 3–17.

BEN-AMRAM, A. AND LEE, C. S. 2007. Program termination analysis in polynomial time. *Trans. Program. Lang. Sys. 29,* 1.

CODISH, M. LAGOON, V. AND STUCKEY, P. 2005. Testing for termination with monotonicity constraints. In *Proceedings of the 21st International Conf. on Logic Programming, (ICLP'05*).

CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., AND STEIN, C. 2001. *Introduction to Algorithms*. MIT Press.

DERSHOWITZ, N. AND MANNA, Z. 1979. Proving termination with multiset orderings. *Comm. ACM 22,* 8, 465–476.

FREDERIKSEN, C. C. 2001. A simple implementation of the size-change principle. Tech. Rep. D-442, DIKU, Denmark.

FREDERIKSEN, C. C. 2002. Automatic runtime analysis for first order functional programs. Tech. Rep. D-470, DIKU, Denmark.

GLENSTRUP, A. AND JONES, N. 2004. Termination analysis and specialization-point insertion in off-line partial evaluation. Tech. Rep. D-498, DIKU, Denmark.

JONES, N. D. AND BOHR, N. 2004. Termination analysis of the untyped lambda calculus. In *Proceedings of the 15th International Conference on Rewriting Techniques and Applications (RTA'04*). Lecture Notes in Computer Science, vol. 3091. Springer, 1–23.

LEE, C. S., JONES, N. D., AND BEN-AMRAM, A. 2001. The size-change principle for program termination. In *Proceedings of the 28th ACM Symposium on Principles of Programming Languages, (POPL'01*). ACM.

MANOLIOS, P. AND VROON, D. 2006. Termination Analysis with Calling Context Graphs. In *Proceedings of the 18th International Conf. on Computer Aided Verification (CAV'06*). Springer, 401–414.

MULLER, D. AND SCHUPP, P. 1995. Simulating Alternating Tree Automata by Nondeterministic Automata: New Results and New Proofs of the Theorems of Rabin, McNaughton and Safra. *Theo. Comput. Sci. 141,* 1&2, 69–107.

SAFRA, S. 1988. On the Complexity of ω-Automata. In *Proceedings of the 29th Symposium on Foundations of Computer Science (FOCS'88*). IEEE, 319–327.

THIEMANN, R. AND GIESL, J. 2003. Size-change termination for term rewriting. In *Proceedings of the 14th International Conf. on Rewriting Techniques and Applications (RTA'03*). Lecture Notes in Computer Science, vol. 2706. Springer, 264–278.

VARDI, M. 1996. An automata-theoretic approach to linear temporal logic. In *Banff Higher Order Workshop*. Lecture Notes in Computer Science, vol. 1043. Springer, 238–266.

WAHLSTEDT, D. 2000. Detecting termination using size-change in parameter values. Master's thesis, Göteborgs Universitet, Sweden.

Received April 2007; revised October 2007; accepted April 2008